MA203 - Numerical Methods
Lecture Notes

Abhinav Jha
Indian Institute of Technology, Gandhinagar
Winter Semester 2026

Preface

These notes are designed to provide a structured and comprehensive understanding of the course content. They will cover key topics, concepts, and computational techniques that are fundamental to numerical methods. Please note that this is the first iteration (Version 0.0.1) of the notes and hence there is a chance that some of the content is incorrect. If you find some flaws, please email me at `abhinav.jha@iitgn.ac.in`.

Obstacles are Opportunities.

—Peacemaker

Contents

1 Floating Point Arithmetic **7**
1.1 Binary and Decimal Number System . . . 9
1.1.1 Normalized Scientific Notation . . . 11
1.1.2 IEEE Notation . . . 11
1.2 Errors . . . 13
1.2.1 Measure of Error . . . 13
1.2.2 Arithmetic Errors . . . 14

2 Root-Finding Methods **19**
2.1 Bracketing Methods . . . 20
2.1.1 Bisection Method . . . 20
2.1.2 Regular-Falsi Method . . . 25
2.2 Open Domain Methods . . . 26
2.2.1 Fixed-Point Iterations . . . 26
2.2.2 Newton-Raphson Method . . . 32
2.3 Error Analysis . . . 36
2.4 Summary . . . 39

3 System of Equations **43**
3.1 Direct Methods . . . 44
3.1.1 Gaussian Elimination . . . 44
3.1.2 Gauss–Jordan Algorithm . . . 50
3.2 Matrix Factorisation . . . 51
3.2.1 LU Decomposition . . . 51
3.2.2 $\mathrm{LDL}^\top$ & Cholesky Decomposition . . . 56
3.3 Iterative Methods . . . 62
3.3.1 Jacobi Method . . . 64
3.3.2 Gauss Seidel Method . . . 66
3.3.3 Error Analysis . . . 69
3.3.4 Condition Number . . . 72
3.4 Summary . . . 74

4 Interpolation **77**
4.1 Lagrange Interpolation . . . 79
4.1.1 Drawbacks . . . 83
4.1.2 Runge Phenomenon . . . 83

4.2 Newton Divided Difference Interpolation 84
4.2.1 Computational Complexity 88
4.3 Summary 88

List of Mathematical Notations **91**

Chapter 1

Floating Point Arithmetic

In engineering sciences, there will be times when one has to solve equations, interpolate data, or compute certain quantities of interest. Finding the (exact) solution of an equation (or any of the aforementioned tasks) is, in principle, the job of mathematics. However, whenever a mathematician sees an equation, instead of directly solving it, we first ask two very important questions:

1. **Existence**: Does a solution exist?
2. **Uniqueness**: If a solution exists, is it unique?

Let us look at two very simple examples:
Example 1:

$$\sum_{n=1}^{\infty} \frac{1}{n}$$

and
Example 2:

$$\sum_{n=1}^{\infty} \frac{1}{n^2}.$$

If you ask any undergraduate student who has studied Calculus, they would say that the first series diverges, whereas the second series converges. This is due to the fact that a p-series, that is,

$$\sum_{n=1}^{\infty} \frac{1}{n^p},$$

converges for $p \in \mathbb{Z}$ with $p > 1$. The idea of convergence of a series was introduced by the French mathematician Baron Augustin-Louis Cauchy (fun fact: his name is inscribed on the Eiffel Tower).

Most areas of mathematics are content with the notions of existence and uniqueness of a solution, but not all. See, even if we have existence and uniqueness, the real challenge is to actually *compute* the solution. In certain cases, we do not even have proofs of existence and uniqueness, but we still need the solution. A classic example of this is the three-dimensional

Figure 1.1: Baron Augustin-Louis Cauchy (21 August 1789 – 23 May 1857).

Navier–Stokes equations. In fact, if you prove existence and uniqueness for this problem, you can win a prize of one million dollars. But if you work in Computational Fluid Dynamics, you already know how important these equations are.

Now, analytical solutions do not always exist, and even if they do, obtaining them in closed form can be extremely difficult. For example, the solution of an ordinary differential equation (ODE) can sometimes be expressed using a power series, but since such series are infinite, implementing them in practice is tough. Also, humans are impatient by nature, and we prefer to delegate the job of computing solutions to computers.

Let us look at Example 2 again and compute the error and the time taken to evaluate the partial sum for different values of n, as shown in Table 1.1.

n	Approximate Sum	Error	Time (s)
10^1	1.54976773	9.52×10^{-2}	0.0000
10^2	1.63498390	9.95×10^{-3}	0.0000
10^3	1.64393457	1.00×10^{-3}	0.0002
10^4	1.64483407	1.00×10^{-4}	0.0024
10^5	1.64492407	1.00×10^{-5}	0.0271
10^6	1.64493307	1.00×10^{-6}	0.2756
10^7	1.64493397	1.00×10^{-7}	2.8460

Table 1.1: Convergence of the series $\sum_{k=1}^{n} \frac{1}{k^2}$ towards $\pi^2/6$.

The true solution of this problem is

$$\frac{\pi^2}{6},$$

which one obtains using Fourier analysis, where this problem is known as the Basel problem. If we look at Table 1.1 carefully, we notice that we obtain a reasonably good approximation of the true solution within seconds. This is the power of numerical methods. However, also notice that after a certain value of n, the reduction in error is not very significant, while the time required increases drastically.

From now until the end of your engineering life, whenever you solve a mathematical problem, there is a 99% chance that you will need to use a numerical method.

I would like to make a clear distinction between two topics: *Numerical Methods* and *Numerical Analysis*. The development of a numerical method may sometimes lead to a false

sense of confidence. There can be situations where a method works for 9 out of 10 problems, but fails for the 10th one. Such methods can be dangerous.

A very infamous example of this is the Tacoma Narrows Bridge collapse of 1940. The bridge collapsed within four months of its opening, the primary reason being a lack of proper structural and eigenvalue analysis during the design phase.

Figure 1.2: Collapse of the Tacoma Narrows Bridge.

Numerical analysis is the branch of mathematics that studies these methods, analyzes their failure rates through error analysis, and tries to make them computationally efficient. This course will only lightly touch upon ideas from numerical analysis and will not go into great depth.

The topic *Numerical Methods* has the word "numerical" in it, which means that we want to solve equations numerically, that is, using calculators or computers. The numbers you have worked with until now belong to the decimal number system. However, computers work using the binary number system. As a result, whenever we solve a problem on a computer, numbers must be converted from one system to another, which inevitably introduces errors.

That is why studying errors is important. In this chapter, we will deal with number systems and the different kinds of errors that we encounter on computers.

1.1 Binary and Decimal Number System

In mathematics, we usually work with the *decimal number system*, also called the *base* 10 *system*. In this system, numbers are written as powers of 10. For example,

$$619.916 = 6 \times 10^2 + 1 \times 10^1 + 9 \times 10^0 + 9 \times 10^{-1} + 1 \times 10^{-2} + 6 \times 10^{-3}.$$

Since everything is expressed in powers of 10, we call this the base 10 number system. If we allow infinitely many digits to the right of the decimal point, then we can represent irrational numbers such as π or e.

Computers, however, do not work with the decimal number system. Instead, they use the *binary number system*, also known as the *base* 2 *system*, which consists of only two digits: 0 and 1. In this system, numbers are written as powers of 2.

Let us briefly discuss how to convert numbers from the decimal system to the binary system and vice versa.

Conversion from decimal to binary

To convert a *whole number* from decimal to binary, we repeatedly divide the number by 2 and record the remainders. The binary representation is obtained by reading the remainders from bottom to top.

Example 1.1. Let us take

$$x = 25_{10},$$

where the subscript $_{10}$ indicates that the number is written in base 10.
We perform successive division by 2:

Division	Quotient	Remainder
25/2	12	1
12/2	6	0
6/2	3	0
3/2	1	1
1/2	0	1

Reading the remainders from bottom to top, we obtain

$$25_{10} = 11001_2.$$

To verify this, we write the binary number using powers of 2:

$$11001_2 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 16 + 8 + 1 = 25.$$

Binary representation of fractional numbers

For numbers less than 1, we use negative powers of 2. To convert a decimal fraction to binary, we repeatedly multiply the number by 2 and record the integer part at each step.

Example 1.2. Let us convert

$$0.125_{10}$$

to binary.

Multiplication	Result	Integer Part
0.125×2	0.25	0
0.25×2	0.50	0
0.50×2	1.00	1

Reading the integer parts from top to bottom, we obtain

$$0.125_{10} = 0.001_2.$$

Combining both parts, we conclude that

$$25.125_{10} = 11001.001_2.$$

This example already hints at an important issue: while some decimal numbers have a finite binary representation, many do not. This fact plays a crucial role in understanding numerical errors on computers, which we will study next.

1.1.1 Normalized Scientific Notation

The numbers we work with can quickly become very large or very small, making them inconvenient to write and store. To deal with this, we use *scientific notation*. In the decimal number system, any non-zero real number can be written in the form

$$x_{10} = \pm r \times 10^n,$$

where

$$0.1 \leq r < 1 \quad \text{and} \quad n \in \mathbb{Z}.$$

If $x = 0$, then we simply take $r = 0$.

Example 1.3. The number

$$1301.2022$$

can be written in scientific notation as

$$1301.2022 = 0.13012022 \times 10^4.$$

In exactly the same way, we can use scientific notation in the binary number system. A non-zero binary number can be written as

$$x_2 = \pm q_2 \times 2^m,$$

where

$$0.1_2 \leq q_2 < 1.0_2 \quad \text{and} \quad m \in \mathbb{Z}.$$

If $x_2 = 0$, then we take $q_2 = 0$.

In both the decimal and binary scientific notations, the quantity r (or q_2) is called the *mantissa*, and the power n (or m) is called the *exponent*.

1.1.2 IEEE Notation

To store binary numbers in a computer, IEEE published a unified framework in 1985 [4], and later released an updated version in 2008 [5]. This standard defines formats for storing numbers in single, double, and extended precision, and is followed by virtually all modern computers.

In the double-precision format, a total of 64 bits are used to represent a real number. The first bit is the *sign bit*, denoted by s, which is 0 for positive numbers and 1 for negative numbers. This is followed by an 11-bit exponent c, called the *characteristic*, and a 52-bit binary fraction f, called the *mantissa*. The base of the exponent is 2.

A schematic representation of this storage format is shown in Fig. 1.3.

Now, we have allocated 52 bits for the mantissa, and hence it can represent numbers up to a precision of 2^{-52}. If we express this precision in the decimal number system, it corresponds to a quantity of the form 10^{-d}. Therefore,

$$10^{-d} = 2^{-52}.$$

Sign	Exponent	Mantissa
s	c	f
1 bit	11 bits	52 bits

Figure 1.3: Visual representation of the IEEE double-precision (64-bit) floating-point storage format.

Taking $\log_{10}$ on both sides, we obtain

$$\begin{aligned} \log_{10}(10^{-d}) &= \log_{10}(2^{-52}), \\ -d &= -52\log_{10}(2), \\ d &\approx 15.65 \approx 16. \end{aligned}$$

Thus, a double-precision floating-point number can represent approximately 16 decimal digits accurately.

The exponent field consists of 11 bits, which allows for $2^{11} = 2048$ possible values. However, not all of these values are used to represent actual exponents. To allow both positive and negative exponents, the exponent is stored in a *biased* form. In double precision, the bias is 1023.

As a result, the actual exponent ranges approximately from -1023 to 1024. With this convention, a normalized floating-point number is represented as

$$(-1)^s 2^{c-1023}(1+f),$$

where

$$0 \leq c \leq 2047,$$

s is the sign bit, c is the stored exponent (characteristic), and f is the fractional part of the mantissa.

Not every real number can be represented in this form. If a computed number is too large to fit within this range, it results in an *overflow* error. Similarly, if a computed number is too small in magnitude to be represented as a normalized number, it results in an *underflow*. Underflow numbers are typically set to zero.

Finally, let us relate this representation to the normalized scientific notation introduced earlier. Recall that we wrote a binary number as

$$x_2 = \pm q_2 \times 2^m,$$

where

$$0.1_2 \leq q_2 < 1.0_2.$$

In the IEEE representation,

$$q_2 = \frac{1+f}{2} \quad \text{and} \quad m = c - 1022.$$

Thus, the IEEE floating-point format is simply a carefully designed way of storing the mantissa q_2 and exponent m in binary, while balancing range, precision, and computational efficiency.

1.2 Errors

We have discussed the conversion of numbers and the errors it can produce. Now, we will concentrate on the decimal number system. Since real-life numbers can have infinitely many digits, but a computer has finite storage, errors can occur while storing numbers.

The errors produced when a calculator or a computer is used to perform real-number calculations are called *round-off errors*. Any positive real number within the numerical range of a machine can be represented as

$$y = 0.d_1 d_2 \dots d_k d_{k+1} \cdots \times 10^n.$$

The floating-point representation of y, denoted by $f_\ell(y)$, is obtained by terminating the mantissa of y after k decimal digits.

This termination can happen in two ways:

1. **Chopping**: In this method, we simply chop off all digits after the k-th digit, that is, the digits $d_{k+1}, d_{k+2}, \dots$ are discarded.

2. **Rounding**: In this method, if $d_{k+1} < 5$, we keep

$$f_\ell(y) = 0.d_1 d_2 \dots d_k \times 10^n.$$

If $d_{k+1} \geq 5$, then we increase the k-th digit by one and write

$$f_\ell(y) = 0.d_1 d_2 \dots (d_k + 1) \times 10^n.$$

For example, let us take the number π. We have

$$\pi = 0.3141592 \cdots \times 10^1.$$

If we use five-digit floating-point arithmetic, then:

- using chopping,

$$f_\ell(\pi) = 0.31415 \times 10^1,$$

- using rounding,

$$f_\ell(\pi) = 0.31416 \times 10^1.$$

1.2.1 Measure of Error

After understanding the different kinds of errors, we now look at how to *measure* these errors.

Definition 1.1. Let p^* be an approximation of p. The *absolute error* is defined as

$$|p - p^*|,$$

and the *relative error* is defined as

$$\frac{|p - p^*|}{|p|},$$

provided that $p \neq 0$.

If the quantity p has physical dimensions (for example, meters or kilograms), then the absolute error has the same dimension, whereas the relative error is dimensionless. Both types of error have their advantages and disadvantages.

Let us first look at relative error. Suppose we have two wooden rods of lengths 1 m and 100 m, and we measure their lengths as 2 m and 101 m, respectively. In both cases, the absolute error is 1 m. However, the relative errors are

$$\frac{1}{1} = 100\% \quad \text{and} \quad \frac{1}{100} = 1\%,$$

respectively. This shows that absolute error alone can be misleading.

On the other hand, if the quantity being approximated is very close to zero, the relative error may become extremely large or even meaningless. In such cases, the absolute error is more informative.

We now introduce the concept of significant digits.

Definition 1.2. The number p^* is said to approximate p to t *significant digits* if t is the largest non-negative integer such that

$$\frac{|p - p^*|}{|p|} \leq 5 \times 10^{-t}.$$

Example 1.4. Suppose we approximate 0.5 by 0.501. Then the relative error is

$$\frac{|0.5 - 0.501|}{0.5} = \frac{0.001}{0.5} = 0.002.$$

We now determine the largest integer t such that

$$0.002 \leq 5 \times 10^{-t}.$$

This gives

$$\begin{aligned} 10^{-t} &\geq 4 \times 10^{-4}, \\ t &\leq \log_{10}(2500), \\ t &\approx 3. \end{aligned}$$

Hence, the approximation is correct up to three significant digits.

1.2.2 Arithmetic Errors

Apart from round-off errors, we may also encounter errors while performing arithmetic operations. Let x and y be two real numbers whose floating-point representations are denoted by $f_\ell(x)$ and $f_\ell(y)$. Let $\oplus$, $\ominus$, $\otimes$, and $\oslash$ denote machine addition, subtraction, multiplication, and

division, respectively. Then finite-point arithmetic is defined as

$$\begin{aligned} x \oplus y &= f_\ell\left(f_\ell(x) + f_\ell(y)\right), \\ x \ominus y &= f_\ell\left(f_\ell(x) - f_\ell(y)\right), \\ x \otimes y &= f_\ell\left(f_\ell(x) \times f_\ell(y)\right), \\ x \oslash y &= f_\ell\left(f_\ell(x) \div f_\ell(y)\right). \end{aligned}$$

Example 1.5. Let

$$x = \frac{5}{7}, \qquad y = \frac{1}{3}.$$

Suppose we use five-digit floating-point arithmetic with chopping.
First, we write both numbers in normalized decimal scientific notation and apply chopping:

$$x = 0.71428 = 0.71428 \times 10^0,$$

$$y = 0.33333\ldots = 0.33333 \times 10^0.$$

Hence,

$$f_\ell(x) = 0.71428 \times 10^0, \qquad f_\ell(y) = 0.33333 \times 10^0.$$

Now we compute

$$\begin{aligned} x \oplus y &= f_\ell\left((0.71428 + 0.33333) \times 10^0\right) \\ &= f_\ell(1.04761 \times 10^0) \\ &= 0.10476 \times 10^1. \end{aligned}$$

The exact value is

$$x + y = \frac{5}{7} + \frac{1}{3} = \frac{22}{21} = 1.04761904\ldots$$

Hence, the absolute error is

$$|(x+y) - (x \oplus y)| \approx 0.19 \times 10^{-4},$$

and the relative error is

$$\frac{|(x+y) - (x \oplus y)|}{|x+y|} \approx 0.182 \times 10^{-4}.$$

The reason we study such errors is that due to finite-point arithmetic, computers may accumulate numerical errors. If an algorithm is applied recursively, a small error at an early stage can grow rapidly and lead to large inaccuracies later. This phenomenon is referred to as *numerical instability*.

Example 1.6. Consider the sequence of real numbers defined by

$$\begin{aligned} x_0^{\mathrm{R}} &= 1, \qquad x_1^{\mathrm{R}} = \frac{1}{3}, \\ x_{n+1}^{\mathrm{R}} &= \frac{13}{3} x_n^{\mathrm{R}} - \frac{4}{3} x_{n-1},^{\mathrm{R}} \qquad (n \geq 1). \end{aligned}$$

It is easy to verify that the exact solution of this recurrence relation is

$$x_n^{\mathrm{T}} = \left(\frac{1}{3}\right)^n.$$

Now, let us generate the sequence numerically using the recurrence relation and compare it with the exact values and setting the rounding to 7 significant digits. Table 1.2 shows the results for the first 15 values of n. We observe that as n increases, the error also increases. Hence, the algorithm is numerically unstable: a small perturbation in the initial values leads to large errors in later iterations.

n	x_n^{R}	x_n^{T}	Error
0	1.0000000	1.0000000	0.0000000
1	0.3333333	0.3333333	0.0000000
2	0.1111110	0.1111111	0.0000001
3	0.0370366	0.0370370	0.0000004
4	0.0123439	0.0123457	0.0000018
5	0.0041081	0.0041152	0.0000071
6	0.0013432	0.0013717	0.0000285
7	0.0003431	0.0004572	0.0001141
8	-0.0003042	0.0001524	0.0004566
9	-0.0017757	0.0000508	0.0018265
10	-0.0072891	0.0000169	0.0073060
11	-0.0292185	0.0000056	0.0292241
12	-0.1168947	0.0000019	0.1168966
13	-0.4675857	0.0000006	0.4675863
14	-1.8703451	0.0000002	1.8703453
15	-7.4813812	0.0000001	7.4813813

Table 1.2: Computed values, exact values, and absolute error

The above example motivates the concept of *conditioning*, which we will discuss later when solving systems of equations. Informally speaking, conditioning measures how sensitive the output of a problem is to small changes in the input.

Until now, the errors we have encountered are mainly due to the computer itself: how numbers are stored and how arithmetic operations are performed on them. However, errors can also arise due to the approximation of mathematical expressions.

Let us consider the Taylor series expansion of a function $f(x) \in \mathcal{C}^\infty[a, b]$ about the

point x. Then,

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2!}f^{(2)}(x) + \cdots + \frac{h^n}{n!}f^{(n)}(x) + \cdots .$$

As we will learn later, the Taylor series is a powerful tool that allows us to construct numerical methods by approximating a function using only the first few terms of its expansion. However, since we are working with finite memory and finite computational resources, we must truncate the Taylor series after a finite number of terms.

The error introduced by neglecting the remaining terms of the series is called the *truncation error*. This topic will be revisited in detail when we study the numerical solution of ordinary and partial differential equations.

Chapter 2

Root-Finding Methods

In mechanical engineering sciences, the notion of *equilibrium* is very important. A system is said to be in mechanical equilibrium if the net force acting on it is zero.

Suppose we have a ball of mass m (kg) freely falling through air. Two forces act on the ball. The first is the gravitational force, given by

$$F_{\mathrm{G}} = mg,$$

where g ($\mathrm{m\,s^{-2}}$) is the acceleration due to gravity. At the same time, the object experiences a drag force given by

$$F_{\mathrm{D}} = kv^2,$$

where k ($\mathrm{kg\,m^{-1}}$) is the drag coefficient, which depends on factors such as the shape of the object, its cross-sectional area, and the air density, and v ($\mathrm{m\,s^{-1}}$) is the speed of the object.

At the maximum constant speed that the object can attain, called the *terminal velocity*, the net force acting on the object is zero. This situation is illustrated in Fig. 2.1. Thus,

$$F_{\mathrm{G}} - F_{\mathrm{D}} = 0 \quad \Rightarrow \quad mg - kv^2 = 0.$$

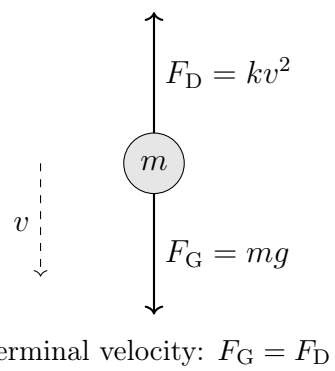

Figure 2.1: Forces acting on a falling object at terminal velocity.

The natural question is: what is the terminal velocity? If all parameters are constant, then the terminal velocity is given by

$$v = \sqrt{\frac{mg}{k}}.$$

However, in real life things are rarely this simple. For example, the drag coefficient k may be a nonlinear function of the velocity, especially if it depends on the Reynolds number[1]. In such a case, the force balance at terminal velocity becomes

$$mg - k(v)v^2 = 0,$$

where k itself depends on v. Finding the solution of such a nonlinear equation is difficult, and we must rely on approximation techniques or numerical root-finding methods.

Even if we restrict ourselves to polynomials, difficulties arise. For a quadratic equation of the form

$$ax^2 + bx + c = 0,$$

the solutions are given explicitly by

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

Closed-form solutions also exist for cubic and bi-quadratic equations. However, it is well known that such closed-form solutions do not exist for general polynomials of degree five or higher, as stated by the Abel–Ruffini theorem.

In this chapter, we study one of the most fundamental and important topics in numerical methods: *root-finding methods*. This is among the earliest numerical ideas known to humanity. For example, as early as 1700 BC, the Babylonians approximated

$$\sqrt{2} \approx 1.414222.$$

2.1 Bracketing Methods

2.1.1 Bisection Method

We begin with the most basic concept in numerical methods: the *Bisection Method*. Before introducing the method, let us recall an important theorem from Real Analysis.

Theorem 2.1 (Intermediate Value Theorem). [1, Theorem 5.3.7] *If $f \in \mathcal{C}[a, b]$ and $k \in [f(a), f(b)]$, then there exists a $c \in [a, b]$ such that*

$$f(c) = k.$$

What this theorem tells us is the following: if a continuous function f takes a value k somewhere between $f(a)$ and $f(b)$, then there exists a point $c \in (a, b)$ such that $f(c) = k$. Now, what if $k = 0$? This means that f has a zero in the interval $[a, b]$. Hence, the Intermediate Value Theorem guarantees the existence of a root.

For the bisection method, we need certain assumptions. The most important one is that

$$f(a)\, f(b) < 0,$$

[1] *Reynolds number* is a dimensionless quantity used to predict flow patterns in different fluid-flow situations

that is, $f(a)$ and $f(b)$ have opposite signs. The remaining assumptions are the same as those required for the Intermediate Value Theorem, namely continuity of f on $[a, b]$.

The basic algorithm (see Fig. 2.2 for a graphical illustration) is as follows:

1. Set $a_1 = a$, $b_1 = b$, and compute
$$p_1 = \frac{a_1 + b_1}{2}.$$
2. If $f(p_1) = 0$, then **STOP**.
3. If $f(p_1)$ and $f(a_1)$ have the same sign, set
$$a_2 = p_1, \qquad b_2 = b_1.$$
4. If $f(p_1)$ and $f(b_1)$ have the same sign, set
$$a_2 = a_1, \qquad b_2 = p_1.$$
5. Repeat the above steps until convergence is achieved or a maximum number of iterations is reached (to avoid an infinite loop).

The algorithm for the bisection method is given in Algorithm 1.

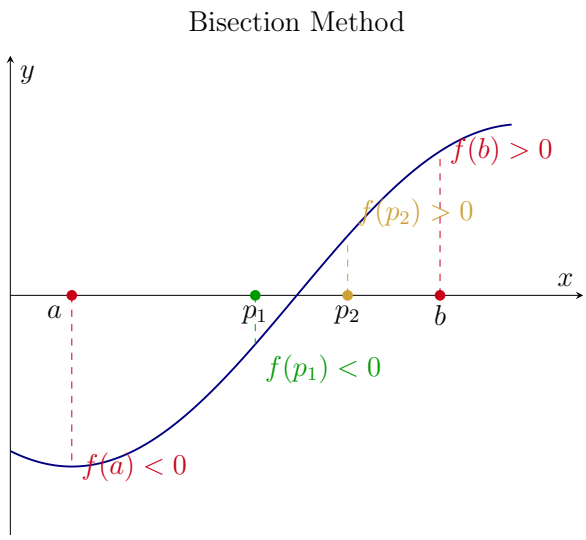

Figure 2.2: Graphical representation of the Bisection Method.

There are several ways to decide when to stop the iteration. In this course, we will use the relative error between successive approximations as the stopping criterion:
$$\frac{|p_n - p_{n-1}|}{|p_n|} < \varepsilon, \qquad p_n \neq 0.$$

Remark 2.1. In Algorithm 1 we notice that we have an **and** condition. Now, we can have situations where $|f(p_n)| <$ `tol` but the function is not close to zero. For example say $f(x) = (x-1)^{10}$, then if $p_k = 0.8$ for some k, we get $|f(p_k)| = 1.024 \times 10^{-7}$. Here if we set `tol` $= 10^{-6}$ then we get convergence if we have **or** instead of **and**.

Algorithm 1 Bisection Method

Given: Continuous function $f(x)$, interval $[a, b]$, tolerance ε, maximum iterations n
Find: Approximation p to a root of f
  Step 1: Check the Existence of a Root
  if $f(a)f(b) \geq 0$ **then**
    Output("Root does not exist in $[a, b]$")
    exit()
  end if
  Step 2: Iterative Refinement
  for $i = 1$ **to** n **do**
    Step 2.1: Compute the Midpoint
    $p = \dfrac{a+b}{2}$
    Step 2.2: Check Stopping Criteria
    if $|f(p)| < \varepsilon$ **and** $\dfrac{b-a}{2} < \varepsilon$ **then**
      return p
    end if
    Step 2.3: Update the Interval
    if $f(a)f(p) < 0$ **then**
      $b = p$
    else
      $a = p$
    end if
  end for
  Step 3: Maximum Iterations Reached
  Output("Maximum number of iterations reached")
  return p

It is important to note that the bisection method always converges to a solution, provided the initial assumptions are satisfied.

Theorem 2.2. *Suppose that $f \in \mathcal{C}[a, b]$ and*

$$f(a)\, f(b) < 0.$$

Then the bisection method generates a sequence $\{p_n\}_{n=1}^{\infty}$ that converges to a zero p of f and satisfies

$$|p_n - p| \leq \frac{b-a}{2^n}, \qquad \text{for } n \geq 1.$$

Proof. We begin with the initial interval $[a_1, b_1] = [a, b]$, whose length is

$$b_1 - a_1 = b - a.$$

At each step of the bisection method, the interval is halved. After the first iteration,

the new interval $[a_2, b_2]$ is either

$$\left[a, \frac{a+b}{2}\right] \quad \text{or} \quad \left[\frac{a+b}{2}, b\right],$$

and hence its length is

$$b_2 - a_2 = \frac{b-a}{2}.$$

Proceeding inductively, after n iterations the interval $[a_n, b_n]$ has length

$$b_n - a_n = \frac{b-a}{2^{n-1}}.$$

By construction of the bisection method, the root p lies inside this interval:

$$p \in (a_n, b_n).$$

The midpoint at the n-th step is

$$p_n = \frac{a_n + b_n}{2}.$$

Since p lies between a_n and b_n, the maximum possible distance between p_n and p is half the length of the interval. Therefore,

$$\begin{aligned} |p_n - p| &\leq \frac{1}{2}(b_n - a_n) \\ &= \frac{1}{2}\frac{b-a}{2^{n-1}} \\ &= \frac{b-a}{2^n}. \end{aligned}$$

This completes the proof. □

Remark 2.2. The estimate above shows that the sequence $\{p_n\}$ converges to p at a rate of $\mathcal{O}\left(\frac{1}{2^n}\right)$. That is, the error is reduced by a factor of 2 at each iteration.

Example 2.1. Determine the number of iterations required to solve

$$f(x) = x^3 + 4x^2 - 10$$

on the interval $[1, 2]$ with an accuracy of 10^{-3}.
From the error bound for the bisection method,

$$|p_n - p| \leq \frac{b-a}{2^n}.$$

Here $b - a = 1$, so we require

$$\frac{1}{2^n} < 10^{-3}.$$

Taking $\log_{10}$ on both sides gives

$$\begin{aligned} -n\log_{10}(2) &< -3, \\ n &> \frac{3}{\log_{10}(2)} \\ &\approx 9.96. \end{aligned}$$

Hence, at least $n = 10$ iterations are required.

Example 2.2. Let

$$f(x) = \sqrt{x} - \cos x$$

on the interval $[0, 1]$.
We compute

$$f(0) = -1, \qquad f(1) \approx 0.459697,$$

so $f(0)f(1) < 0$ and the bisection method applies.

- **Iteration 1:**

$$p_1 = \frac{0+1}{2} = 0.5,$$
$$f(p_1) \approx -0.170475 < 0.$$

  Hence, the new interval is $[0.5, 1]$.

- **Iteration 2:**

$$p_2 = \frac{0.5+1}{2} = 0.75,$$
$$f(p_2) \approx 0.134336 > 0.$$

  Hence, the new interval is $[0.5, 0.75]$.

Continuing in this manner, the interval containing the root keeps shrinking and the midpoint converges to the solution.

Remark 2.3. The choice of the starting interval plays an important role in the success of the bisection method. If the interval is too large, it may contain multiple roots. Therefore, to locate a specific root, we must choose an appropriate initial interval.

Graphing the function often helps in selecting such an interval. This approach is commonly referred to as a *graphical method*. While graphical methods are useful for locating roots approximately, they are not always reliable for accurate computations.

2.1.2 Regular-Falsi Method

Now, while studying the bisection method, we noticed that we did not use any property of the function $f(x)$ in the iteration process except the sign of the function. The *regular falsi* method (also known as the *false position* method) is another bracketing approach in which we use both the sign and the value of the function.

We assume that the assumptions of the bisection method are satisfied. Let

$$a_1 = a, \qquad b_1 = b,$$

with

$$f(a_1)\, f(b_1) < 0.$$

The equation of the straight line joining the points $(a_1, f(a_1))$ and $(b_1, f(b_1))$ is given by

$$y - f(a_1) = \frac{f(b_1) - f(a_1)}{b_1 - a_1}(x - a_1).$$

The x-intercept of this line is obtained by setting $y = 0$, which gives

$$x = a_1 - f(a_1)\,\frac{b_1 - a_1}{f(b_1) - f(a_1)}.$$

We denote this value by p_1. Depending on the sign of $f(p_1)$, we choose the new interval in the same way as in the bisection method (see Fig. 2.3). Repeating this process generates a sequence that converges to the root.

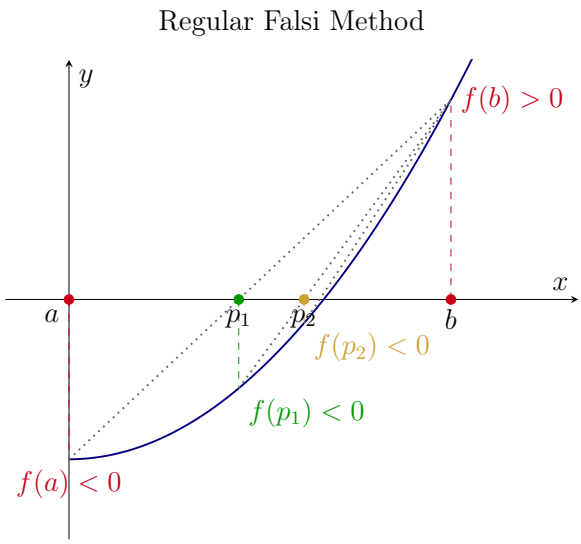

Figure 2.3: Graphical representation of the Regular Falsi.

Remark 2.4. If the root of the equation lies close to one endpoint of the interval, the regular falsi method can converge faster than the bisection method, since it avoids unnecessary halving of the interval.

Example 2.3. Let

$$f(x) = \sqrt{x} - \cos x$$

on the interval $[0, 1]$.
We compute

$$f(0) = -1, \qquad f(1) \approx 0.459697,$$

so $f(0)f(1) < 0$ and the regular falsi method applies.

- **Iteration 1:**

$$p_1 = 0 - (-1)\frac{1-0}{0.459697 - (-1)} \approx 0.6850733.$$

  Since,

$$f(p_1) \approx 0.0533189 > 0.$$

  Hence, the new interval is [0, 0.6850733].

- **Iteration 2:**

$$p_2 = 0 - (-1)\frac{0.6850733 - 0}{0533189 - (-1)} \approx 0.6503946.$$

$$f(p_2) \approx 0.0101 > 0.$$

  Hence, the new interval is [0, 0.6503946].

Continuing in this manner, the approximations move rapidly toward the root, which lies close to the left endpoint of the interval.

Example 2.4. Consider the function

$$f(x) = x^2 - 6$$

on the interval $[2, 4]$.
We plot the relative error obtained using both the bisection method and the regular falsi method in Fig. 2.4. As the number of iterations n increases, we observe that the relative error for the regular falsi method decreases more rapidly than that of the bisection method. In particular, the error for the regular falsi method is consistently smaller.
This behavior occurs because the root of the equation lies closer to one endpoint of the interval. In such situations, the regular falsi method benefits from using the function values and moves the approximation more effectively toward the root, whereas the bisection method continues to halve the interval symmetrically.

2.2 Open Domain Methods

The methods that we have seen until now relies heavily on the interval in which the root lies but we have methods that only require a starting point and are known as open domain methods.

2.2.1 Fixed-Point Iterations

Let us recall what a fixed point is.

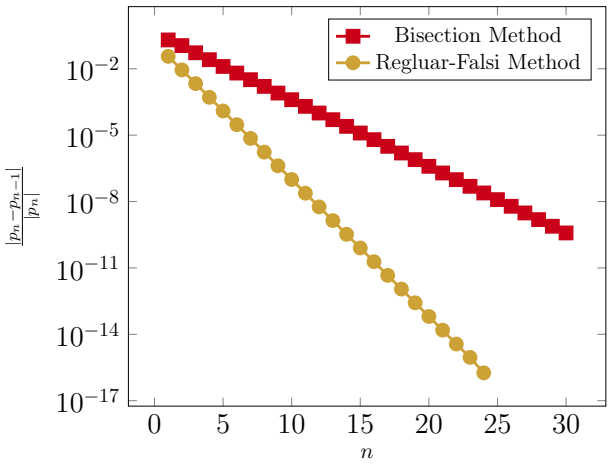

Figure 2.4: Relative error of the bisection method and the regular falsi method applied to $f(x) = x^2 - 6$ on the interval $[2, 4]$.

Definition 2.1 (Fixed Point). A number p is called a *fixed point* of a function g if

$$g(p) = p.$$

Example 2.5.
1. The function $f(x) = \sin x$ has a unique fixed point at $x = 0$.
2. The function $g(x) = e^x$ has no fixed point.
3. The function $h(x) = x^2 - 6$ has two fixed points, -2 and 3, in the interval $[-3, 3]$.

The reason we study fixed-point problems is that there is a close connection between fixed-point methods and root-finding methods.

(RF) Given a root-finding problem: find p such that

$$f(p) = 0,$$

we can construct a fixed-point problem by defining a function g as

$$g(x) = x - f(x) \quad \text{or} \quad g(x) = x - \alpha f(x), \qquad \alpha \neq 0.$$

Any solution p of the fixed-point equation $g(p) = p$ is then a root of f.

Similarly,

(FP) If a function $g(x)$ has a fixed point at p, then the function

$$f(x) = x - g(x)$$

has a zero at p.

We now ask two natural questions: when does a fixed point exist, and when is it unique?

Theorem 2.3 (Existence and Uniqueness of Fixed Points). *1. If $g \in \mathcal{C}[a,b]$ and*

$$g(x) \in [a,b] \quad \forall\, x \in [a,b],$$

then g has at least one fixed point in $[a,b]$.

2. If, in addition, $g'(x)$ exists on (a,b) and there exists a constant $0 < k < 1$ such that

$$|g'(x)| \leq k \quad \forall\, x \in (a,b),$$

then g has exactly one fixed point in $[a,b]$.

Remark 2.5. In Example 2.5, we observed that $g(x) = e^x$ has no fixed point. This happens because e^x does not map any interval $[a,b]$ into itself.

For the function $h(x) = x^2 - 6$, the derivative $h'(x) = 2x$ exists but is not bounded by 1 on the interval $[-3,3]$. Hence, the uniqueness condition fails, and indeed we obtain two fixed points.

It is important to note that the uniqueness condition is sufficient but not necessary. For example, if we restrict $h(x)$ to the interval $[0,2]$, the derivative is still not bounded by 1, yet the function has a unique fixed point in this interval.

We now discuss how to approximate fixed points numerically. This leads to the *fixed-point iteration method*. Given a function $g(x)$ and an initial approximation p_0, we generate a sequence $\{p_n\}_{n=0}^{\infty}$ defined by

$$p_n = g(p_{n-1}), \qquad n \geq 1.$$

If $p_n \to p$ as $n \to \infty$ and g is continuous, then

$$p = \lim_{n\to\infty} p_n = \lim_{n\to\infty} g(p_{n-1}) = g\Big(\lim_{n\to\infty} p_{n-1}\Big) = g(p),$$

and hence p is a fixed point of g (see Fig. 2.5).

Algorithm for the fixed point iteration is given in Algorithm 2.

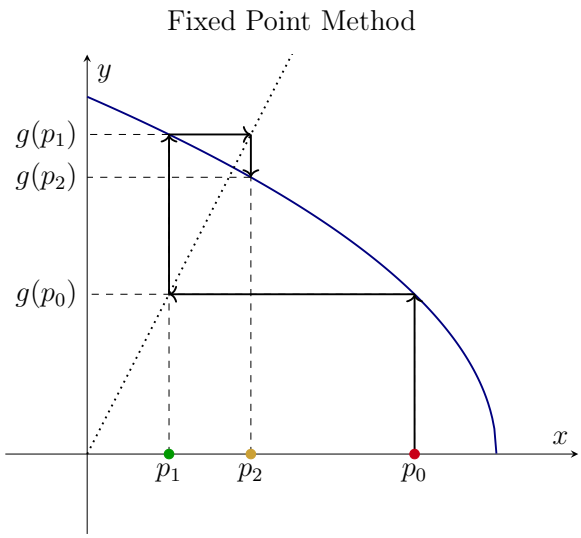

Figure 2.5: Graphical representation of the Fixed Point Method.

Example 2.6. Consider the problem of finding a root of

$$f(x) = x^3 + 4x^2 - 10$$

in the interval $[1, 2]$.
Define the fixed-point function

$$g(x) = x - \frac{x^3 + 4x^2 - 10}{3x^2 + 8x}.$$

Let us choose the initial guess $p_0 = 1.5$.

1. **Iteration 1:**

$$p_1 = g(p_0) = 1.5 - \frac{1.5^3 + 4(1.5)^2 - 10}{3(1.5)^2 + 8(1.5)} \approx 1.37333333.$$

As we do not know p, we compute the relative error as

$$\frac{|p_n - p_{n-1}|}{|p_n|}.$$

Hence, the relative error is

$$\frac{|p_1 - p_0|}{|p_1|} \approx 0.0922233.$$

2. **Iteration 2:**

$$p_2 = g(p_1) \approx 1.36526201.$$

The relative error is

$$\frac{|p_2 - p_1|}{|p_2|} \approx 0.00591.$$

Continuing in this manner, the sequence converges to the desired root.

Algorithm 2 Fixed Point Iteration Method

Given: Function $g(x)$, initial guess p_0, tolerance ε, maximum iterations n
Find: Approximation p such that $p \approx g(p)$
  Step 1: Initialize
  $p = p_0$
  Step 2: Iterative Refinement
  for $i = 1$ **to** n **do**
    Step 2.1: Compute Next Iterate
    $p_{\text{new}} = g(p)$
    Step 2.2: Check Stopping Criterion
    if $|p_{\text{new}} - p| < \varepsilon$ **then**
      return p_{new}
    end if
    Step 2.3: Update Approximation
    $p = p_{\text{new}}$
  end for
  Step 3: Maximum Iterations Reached
  Output("Maximum number of iterations reached")
  return p

In the previous example, we chose the function $g(x)$ in what may appear to be a rather strange way. A natural question now arises: can we choose $g(x)$ in some other way? The answer is yes.

Let us look at two more choices of $g(x)$:

$$g_2(x) = x - x^3 - 4x^2 + 10 \qquad \text{and} \qquad g_3(x) = \frac{1}{2}\sqrt{10 - x^3}.$$

Table 2.1 shows the values of the sequence $\{p_n\}$ obtained using these functions. We observe that $g_2(x)$ does not converge at all (in fact, it diverges), $g_1(x)$ has the fastest convergence, whereas $g_3(x)$ converges, but rather slowly. This naturally raises the question: why does this happen?

To understand this behavior, we need a convergence theorem for fixed-point iteration.

n	$p_n\,(g_1)$	$p_n\,(g_2)$	$p_n\,(g_3)$
0	1.5	1.5	1.5
1	1.3733333333333333	-0.875	1.286953767623375
2	1.3652620148746266	6.732421875	1.4025408035395783
3	1.3652300139161466	-469.72001200169325	1.3454583740232942
4	1.3652300134140969	102754555.18738511	1.3751702528160383
5	1.3652300134140969	$-1.0849338705317464 \times 10^{24}$	1.360094192761733
6	—	$1.277055591444378 \times 10^{72}$	1.3678469675921328
7	—	—	1.3638870038840212
8	—	—	1.36591673339004
9	—	—	1.364878217193677
10	—	—	1.365410061169957

Table 2.1: Comparison of fixed-point iterations for g_1, g_2, and g_3.

Theorem 2.4. *Let $g \in \mathcal{C}[a,b]$ be such that*

$$g(x) \in [a,b] \quad \forall\, x \in [a,b].$$

Suppose, in addition, that $g'(x)$ exists on (a,b) and there exists a constant $0 < k < 1$ such that

$$|g'(x)| \leq k \quad \forall\, x \in (a,b).$$

Then, for any initial guess $p_0 \in [a,b]$, the sequence defined by

$$p_n = g(p_{n-1}), \qquad n \geq 1,$$

converges to the unique fixed point $p \in [a,b]$.

Corollary 2.5. *If $g(x)$ satisfies the hypotheses of Theorem 2.4, then the error in using p_n to approximate p satisfies*

$$|p_n - p| \leq k^n \max\{p_0 - a,\, b - p_0\},$$

and

$$|p_n - p| \leq \frac{k^n}{1-k}\,|p_1 - p_0|, \qquad n \geq 1.$$

Remark 2.6. From the above corollary, we see that the rate of convergence of the sequence $\{p_n\}$ depends on the constant k that bounds $|g'(x)|$. Smaller values of k lead to faster convergence.

Let us now revisit the examples.

1. Consider

$$g_2(x) = x - x^3 - 4x^2 + 10.$$

We have

$$g_2(1) = 6, \qquad g_2(2) = -12,$$

so g_2 does not map $[1, 2]$ into itself. Moreover,

$$g_2'(x) = 1 - 3x^2 - 8x,$$

and hence

$$|g_2'(x)| > 1 \quad \forall\, x \in [1, 2].$$

Both conditions of Theorem 2.4 fail, and as expected, the iteration diverges.

2. Now consider

$$g_3(x) = \frac{1}{2}\sqrt{10 - x^3}.$$

We compute

$$g_3'(x) = -\frac{3}{4}\frac{x^2}{\sqrt{10 - x^3}},$$

which is negative on $[1, 2]$, so g_3 is decreasing on this interval. However,

$$|g_3'(2)| \approx 2.12 > 1,$$

and hence the derivative condition fails on $[1, 2]$.

Nevertheless, numerical experiments show convergence. The reason is that the root lies in the smaller interval $[1, 1.5]$. On this interval,

$$g_3 : [1, 1.5] \to [1, 1.5] \quad \text{and} \quad |g_3'(x)| < 1,$$

so the hypotheses of Theorem 2.4 are satisfied locally.

But why is the convergence of $g_1(x)$ so good?

2.2.2 Newton-Raphson Method

Suppose that $f \in \mathcal{C}^2[a, b]$. Let $p \in (a, b)$ be a root of f, that is,

$$f(p) = 0,$$

and let $p_0 \in [a, b]$ be an initial approximation such that

$$f'(p_0) \neq 0.$$

Consider the Taylor expansion of $f(x)$ about p_0 and evaluate it at p:

$$f(p) = f(p_0) + (p - p_0)f'(p_0) + \frac{(p - p_0)^2}{2!}f''(\xi), \qquad \xi \in (p_0, p).$$

Since $f(p) = 0$ and we assume that p is close to p_0, the higher-order term is small and can be neglected. This gives

$$\begin{aligned} p - p_0 &= -\frac{f(p_0)}{f'(p_0)}, \\ p &= p_0 - \frac{f(p_0)}{f'(p_0)} \equiv p_1. \end{aligned}$$

Continuing in this manner, we generate a sequence $\{p_n\}_{n=0}^{\infty}$ defined by

$$p_n = p_{n-1} - \frac{f(p_{n-1})}{f'(p_{n-1})}, \qquad n \geq 1.$$

This method is called the *Newton–Raphson method*, named after Sir Isaac Newton and Joseph Raphson. Newton developed the idea, and Raphson later refined and published it.

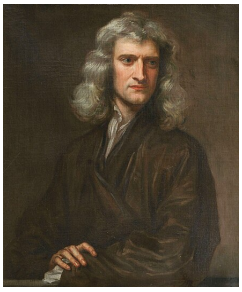

Figure 2.6: Isaac Newton (4 January 1643 – 31 March 1727).

Newton's method can also be viewed as a fixed-point iteration with

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

In the previous section, the function $g_1(x)$ was precisely this choice. Algorithm for the Newton method is given in Algorithm 3.

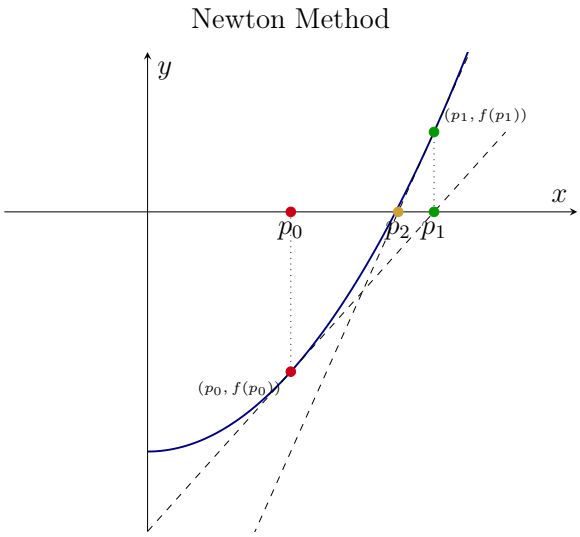

Figure 2.7: Graphical representation of the Newton–Raphson method.

We now discuss the convergence of Newton's method.

Algorithm 3 Newton–Raphson Method

Given: Function $f(x)$, derivative $f'(x)$, initial guess p_0, tolerance ε, maximum iterations n
Find: Approximation p to a root of f
  Step 1: Initialize
  $p = p_0$
  Step 2: Iterative Refinement
  for $i = 1$ **to** n **do**
    Step 2.1: Check Derivative
    if $f'(p) = 0$ **then**
      Output("Derivative zero. Method fails.")
      exit()
    end if
    Step 2.2: Compute Next Iterate
    $p_{\text{new}} = p - \dfrac{f(p)}{f'(p)}$
    Step 2.3: Check Stopping Criteria
    if $|p_{\text{new}} - p| < \varepsilon$ **or** $|f(p_{\text{new}})| < \varepsilon$ **then**
      return p_{new}
    end if
    Step 2.4: Update Approximation
    $p = p_{\text{new}}$
  end for
  Step 3: Maximum Iterations Reached
  Output("Maximum number of iterations reached")
  return p

Theorem 2.6 (Convergence of Newton's Method)**.** *Let $f \in \mathcal{C}^2[a,b]$. Suppose that $p \in (a,b)$ satisfies*

$$f(p) = 0 \quad \textit{and} \quad f'(p) \neq 0.$$

Then there exists a $\delta > 0$ such that Newton's method generates a sequence $\{p_n\}_{n=0}^{\infty}$ converging to p for any initial approximation

$$p_0 \in [p - \delta,\, p + \delta].$$

The above theorem shows that the choice of the initial approximation plays a crucial role. If p_0 is too far from the root p, Newton's method may fail to converge.

Example 2.7. Consider the problem of finding a root of

$$f(x) = x^2 - 6$$

in the interval $[1, 2]$. Let the initial guess be $p_0 = 1$.
Using Newton's method, we obtain:

1. **Iteration 1:**

$$p_1 = p_0 - \frac{f(p_0)}{f'(p_0)} = 1 - \frac{1^2 - 6}{2 \cdot 1} = 3.5.$$

   The relative error is

$$\frac{|p_1 - p_0|}{|p_1|} = \frac{2.5}{1} = 0.7142857.$$

2. **Iteration 2:**

$$p_2 = p_1 - \frac{f(p_1)}{f'(p_1)} = 3.5 - \frac{(3.5)^2 - 6}{2 \cdot 3.5} \approx 2.6071429.$$

   The relative error is

$$\frac{|p_2 - p_1|}{|p_2|} \approx 0.342.$$

Continuing in this manner, the sequence converges to the root $\sqrt{6}$.

In Fig. 2.8 we present the relative errors obtained for x^-6 by using all the three methods. We observe that the Newton method is by far the most efficient method of all the three.

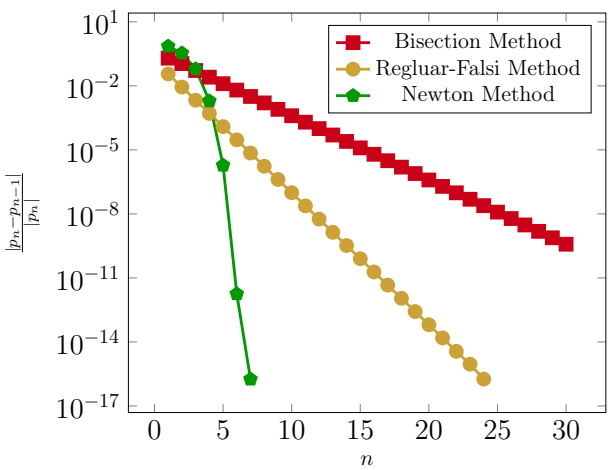

Figure 2.8: Relative error of the bisection method, regular falsi method, and the Newton method applied to $f(x) = x^2 - 6$ on the interval $[2, 4]$.

Remark 2.7. In Newton's method, we observe that the computation of the derivative of the function is required. However, this derivative is not always readily available or easy to compute. In such cases, one can replace the derivative by an approximation.

By definition of the derivative at a point, we have

$$f'(p_{n-1}) = \lim_{x \to p_{n-1}} \frac{f(x) - f(p_{n-1})}{x - p_{n-1}}.$$

If the previous approximation p_{n-2} is sufficiently close to p_{n-1}, we can approximate the derivative as

$$f'(p_{n-1}) \approx \frac{f(p_{n-1}) - f(p_{n-2})}{p_{n-1} - p_{n-2}}.$$

Replacing the derivative in Newton's method by this approximation, we obtain

$$p_n = p_{n-1} - \frac{f(p_{n-1})}{f(p_{n-1}) - f(p_{n-2})}(p_{n-1} - p_{n-2}).$$

This iterative scheme is known as the *Secant Method*, since the new approximation is obtained by drawing the secant line through the points

$$(p_{n-1}, f(p_{n-1})) \quad \text{and} \quad (p_{n-2}, f(p_{n-2})).$$

A keen observer will notice that the secant method is similar in spirit to the regular falsi method. However, unlike regular falsi, the secant method does not maintain a bracketing interval and does not require checking the sign of the function values.

2.3 Error Analysis

We have observed that Newton's method significantly outperforms both the bisection method and the regular falsi method. We have discussed convergence informally so far; let us now give it a precise definition. We begin by recalling the notion of convergence of a sequence.

Definition 2.2 (Convergence of a Sequence)**.** A sequence $\{p_n\}$ is said to converge to a number p if for every $\varepsilon > 0$ there exists an $n_0 \in \mathbb{N}$ such that

$$|p_n - p| < \varepsilon \quad \text{whenever} \quad n \geq n_0.$$

Once we know that a sequence of approximations converges, the next natural question is: how fast does it converge? This leads to the notion of order of convergence.

Definition 2.3 (Order of Convergence)**.** Suppose $\{p_n\}_{n=0}^{\infty}$ is a sequence that converges to p, with $p_n \neq p$ for all n. If there exist positive constants λ and α such that

$$\lim_{n \to \infty} \frac{|p_{n+1} - p|}{|p_n - p|^{\alpha}} = \lambda, \tag{2.1}$$

then the sequence $\{p_n\}$ is said to converge to p with order α and asymptotic error constant λ.

If $\alpha = 1$ and $0 < \lambda < 1$, the convergence is called *linear*. If $\alpha = 2$, the convergence is called *quadratic*.

Example 2.8. Consider the sequence

$$p_n = \frac{1}{2^n}.$$

Clearly, $p_n \to 0$ as $n \to \infty$. The error relation becomes

$$\frac{|p_{n+1} - 0|}{|p_n - 0|^\alpha} = \frac{2^{n\alpha}}{2^{n+1}}.$$

For $\alpha = 1$, we obtain

$$\lim_{n\to\infty} \frac{1}{2} = \frac{1}{2},$$

and hence the sequence converges linearly. For any $\alpha > 1$, the above expression diverges, showing that the convergence is not of higher order.

We now state results that explain the convergence behavior of fixed-point iterations.

Theorem 2.7. *Let $g \in \mathcal{C}[a, b]$ be such that*

$$g(x) \in [a, b] \quad \forall\, x \in [a, b].$$

Suppose, in addition, that g' is continuous on (a, b) and there exists a constant $0 < k < 1$ such that

$$|g'(x)| \leq k \quad \forall\, x \in (a, b).$$

If $g'(p) \neq 0$, then for any $p_0 \neq p$ in $[a, b]$, the sequence

$$p_n = g(p_{n-1}), \quad n \geq 1,$$

converges linearly *to the unique fixed point p in $[a, b]$.*

We will not prove the above theorem here, but it clearly suggests that higher-order convergence is possible only if

$$g'(p) = 0,$$

that is, the derivative of the fixed-point function vanishes at the fixed point. The next result makes this precise.

Theorem 2.8. *Let p be a solution of the equation $x = g(x)$. Suppose that $g'(p) = 0$ and that g'' is continuous on an open interval I containing p, with*

$$|g''(x)| \leq M \quad \forall\, x \in I.$$

Then there exists a $\delta > 0$ such that, for any

$$p_0 \in [p - \delta,\, p + \delta],$$

the sequence defined by

$$p_n = g(p_{n-1}), \quad n \geq 1,$$

converges at least quadratically to p. Moreover, for sufficiently large n,

$$|p_{n+1} - p| \leq \frac{M}{2}|p_n - p|^2.$$

The above two theorems together show that for a fixed-point iteration to converge quadratically, we must have

$$g(p) = p \quad \text{and} \quad g'(p) = 0.$$

As seen earlier, a fixed-point problem can be constructed from a root-finding problem by defining

$$g(x) = x - \varphi(x)f(x),$$

where $\varphi(x)$ is a differentiable function.

To obtain quadratic convergence, we require $g'(p) = 0$. Differentiating $g(x)$ gives

$$g'(x) = 1 - \varphi'(x)f(x) - \varphi(x)f'(x).$$

Using the fact that $f(p) = 0$, we obtain

$$g'(p) = 1 - \varphi(p)f'(p).$$

Thus, choosing

$$\varphi(p) = \frac{1}{f'(p)}$$

ensures that $g'(p) = 0$ and hence quadratic convergence. With this choice, the fixed-point function becomes

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

The resulting iteration

$$p_n = g(p_{n-1}) = p_{n-1} - \frac{f(p_{n-1})}{f'(p_{n-1})}$$

is precisely Newton's method.

To compute the order of convergence numerically, we take the logarithm on both sides of Eq. (2.1). Starting from

$$|p_{n+1} - p| = \lambda|p_n - p|^\alpha,$$

we obtain

$$\log(|p_{n+1} - p|) = \log(\lambda) + \alpha \log(|p_n - p|).$$

Rearranging, we get

$$\alpha = \frac{\log(|p_{n+1} - p|) - \log(\lambda)}{\log(|p_n - p|)}.$$

If λ is unknown or sufficiently small, a practical approximation for the observed order of convergence is

$$\alpha \approx \frac{\log(|p_{n+1} - p|)}{\log(|p_n - p|)}.$$

Example 2.9. Consider the function

$$f(x) = x^2 - 6$$

on the interval $[2, 4]$. The exact root is

$$p = \sqrt{6} \approx 2.44948974326.$$

Table 2.2 shows the observed order of convergence obtained using the bisection method and Newton's method. We observe that the bisection method converges linearly, whereas Newton's method converges quadratically.

Bisection Method			**Newton Method**		
n	Error	Order α	n	Error	Order α
0	5.103×10^{-1}	–	1	1.051×10^{0}	–
1	4.995×10^{-1}	0.092	2	1.577×10^{-1}	−37.49
2	2.495×10^{-1}	2.000	3	4.767×10^{-3}	2.894
3	1.245×10^{-1}	1.501	4	4.629×10^{-6}	2.298
4	6.199×10^{-2}	1.335	5	4.374×10^{-12}	2.129
5	3.074×10^{-2}	1.252	6	0.000×10^{0}	–
6	1.511×10^{-2}	1.204			
7	7.302×10^{-3}	1.174			
8	3.396×10^{-3}	1.156			
9	1.443×10^{-3}	1.151			
10	4.663×10^{-4}	1.173			

Table 2.2: Observed order of convergence for the bisection and Newton methods.

2.4 Summary

In this chapter, we studied different methods for finding the roots of nonlinear equations. Broadly, these methods can be classified into *bracketing methods* and *open-domain methods*. Each class has its own strengths and weaknesses, and the choice of method depends heavily on the nature of the problem.

1. **Bracketing Methods:**

   (a) These methods require an interval $[a, b]$ such that $f(a)$ and $f(b)$ have opposite signs. This is not always possible. For example, the function $f(x) = x^2$ has a root at $x = 0$ but does not change sign.

   (b) Bracketing methods are guaranteed to converge as long as the initial assumptions are satisfied. This makes them very reliable.

   (c) The bisection method provides an explicit upper bound on the error, which allows us to estimate the number of iterations required to achieve a desired accuracy. The regular falsi method does not provide such a simple bound.

   (d) These methods converge slowly. In particular, the bisection method converges linearly and does not make use of the shape of the function.

   (e) Bracketing methods can fail to detect roots of even multiplicity, since the function does not change sign at such roots.

   (f) Bracketing methods are generally robust to poor initial choices, as long as a valid interval is provided.

2. **Open-Domain Methods:**

   (a) These methods do not require an initial interval containing the root. Instead, they require one or more initial guesses.

   (b) Open-domain methods are usually much faster than bracketing methods. For example, Newton's method converges quadratically near the root.

   (c) These methods are not guaranteed to converge. A poor initial guess may lead to divergence or convergence to an unintended root.

   (d) If an equation has multiple roots, open-domain methods may converge to different roots depending on the initial guess.

   (e) These methods often require the computation of derivatives (for example, Newton's method), which may not always be available or easy to compute.

   (f) Open-domain methods are sensitive to rapidly changing derivatives. For instance, for functions like $f(x) = \sin(1/x)$ near $x = 0$, Newton's method can behave unpredictably.

When to Use Which Method?

- If you only know that a root exists and you want a *guaranteed* method, use a **bracketing method**. **Example:** To solve

  $$f(x) = x^3 - 5x - 2,$$

  if you find that $f(2) < 0$ and $f(3) > 0$, the bisection method is a safe choice.

- If you have a good initial guess and the derivative is easy to compute, use an **open-domain method**. **Example:** To compute $\sqrt{6}$ by solving

$$f(x) = x^2 - 6,$$

  Newton's method converges very rapidly once the initial guess is close to the root.

- If the derivative is unavailable or expensive to compute, but you still want faster convergence than bisection, use the **secant method**. **Example:** Solving

$$f(x) = e^{-x} - x$$

  using two initial guesses close to the root.

- If the function is highly nonlinear or behaves badly, it is often safer to *first bracket the root* and then switch to an open method. **Example:** Use the bisection method to narrow down the interval and then apply Newton's method for fast convergence.

Chapter 3

System of Equations

In electrical engineering, one of the most important laws is **Kirchhoff's Voltage Law (KVL)**. In simple terms, this law states that the algebraic sum of all potential differences (voltages) around any closed loop in an electrical circuit is zero.

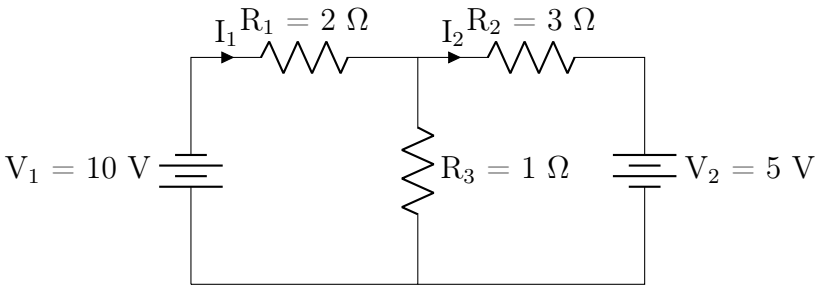

Figure 3.1: Two-loop circuit illustrating Kirchhoff's Voltage Law

Let I_i denote the current in the i-th loop and R_i denote the corresponding resistance. Using Ohm's law, the voltage drop across a resistor can be written as

$$V_i = I_i R_i.$$

Applying Kirchhoff's Voltage Law to the left loop in Fig. 3.1, we obtain

$$10 - 2I_1 - (I_1 - I_2) = 0,$$

which simplifies to

$$-3I_1 + I_2 = -10.$$

Similarly, applying Kirchhoff's Voltage Law to the right loop gives

$$5 - 3I_2 - (I_2 - I_1) = 0,$$

or equivalently,

$$I_1 - 4I_2 = -5.$$

Thus, in order to determine the currents flowing in each loop, we are required to solve a system of linear equations. As the number of loops in a circuit increases, the number of unknown currents and equations also increases. Another important application arises in the discretisation of differential equations, a technique widely used in civil, mechanical, and electrical engineering. When differential equations are discretised, the resulting system of equations often takes the form of a *band matrix*[1]. Depending on the choice of polynomial approximation used in the discretisation, the resulting band matrix can be tridiagonal, pentadiagonal, or a more general band matrix. Efficiently solving these systems is crucial for obtaining numerical solutions to the differential equations.

In this chapter, we first introduce *direct methods* for solving systems of equations, followed by iterative methods. Direct methods aim to compute the exact solution theoretically in a finite number of steps. However, in practical computations, round-off errors are unavoidable and must be carefully managed to ensure numerical accuracy.

3.1 Direct Methods

3.1.1 Gaussian Elimination

The most fundamental method studied in linear algebra for solving a system of linear equations is *Gaussian elimination*. Although the method was developed independently in ancient China and early modern Europe, it became widely known through the work of Carl Friedrich Gauss. The name *Gaussian elimination* was later popularised by George Forsythe.

Figure 3.2: Carl Friedrich Gauss (30 April 1777 – 23 February 1855).

Suppose we have a system of n linear equations in n unknowns of the form

$$\begin{aligned}
\mathrm{R}_1 \quad &: a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &&= b_1, \\
\mathrm{R}_2 \quad &: a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &&= b_2, \\
&\qquad\qquad \vdots && \quad \vdots \\
\mathrm{R}_n \quad &: a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &&= b_n.
\end{aligned} \tag{3.1}$$

[1]**Band Matrix:** A matrix $\{a_{ij}\}_{i,j=1}^{n}$ is called a band matrix if all elements outside a certain diagonal band are zero. The band is determined by

$$a_{ij} = 0 \quad \text{if} \quad j < i - k_1 \quad \text{or} \quad j > i + k_2, \quad k_1, k_2 \geq 0,$$

where k_1 and k_2 are the lower and upper bandwidths, respectively. Special cases include diagonal matrices ($k_1 = k_2 = 0$) and tridiagonal matrices ($k_1 = k_2 = 1$).

In Eq. (3.1), the matrix $\{a_{ij}\}_{i,j=1}^n$ and the vector $\{b_i\}_{i=1}^n$ are given, while $\{x_i\}_{i=1}^n$ are the unknown variables.

These systems of equations satisfy certain properties that allow us to transform them into an equivalent but simpler system, which is easier to solve. We recall three such elementary row operations:

1. **Scalar multiplication:** $\mathrm{R}_i \mapsto \lambda \mathrm{R}_i$, where $\lambda \in \mathbb{R}$ and $\lambda \neq 0$.
2. **Row replacement:** $\mathrm{R}_i \mapsto \mathrm{R}_i + \lambda \mathrm{R}_j$, for some $j = 1, \ldots, n$ with $j \neq i$.
3. **Row interchange:** $\mathrm{R}_i \leftrightarrow \mathrm{R}_j$, for $i \neq j$.

We can represent the system of equations in Eq. (3.1) as an $n \times (n+1)$ matrix $[\mathbf{A}, \mathbf{b}]$, called the *augmented matrix*, given by

$$[\mathbf{A}, \mathbf{b}] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \ldots & a_{1n} & b_1 \\ a_{21} & a_{22} & \ldots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \ldots & a_{nn} & b_n \end{array}\right]. \tag{3.2}$$

The vertical line separates the coefficient matrix $\mathbf{A}$ from the right-hand-side vector $\mathbf{b}$. The idea of Gaussian elimination with backward substitution is to reduce the augmented matrix to an upper triangular form and then solve the resulting system by backward substitution.

For notational convenience, let us denote b_i by $a_{i,n+1}$ for $i = 1, 2, \ldots, n$. Then

$$\tilde{\mathbf{A}} = [\mathbf{A}, \mathbf{b}] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \ldots & a_{1n} & a_{1,n+1} \\ a_{21} & a_{22} & \ldots & a_{2n} & a_{2,n+1} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \ldots & a_{nn} & a_{n,n+1} \end{array}\right]. \tag{3.3}$$

Assuming $a_{11} \neq 0$, we perform the row operations

$$\mathrm{R}_j \mapsto \mathrm{R}_j - \frac{a_{j1}}{a_{11}} \mathrm{R}_1, \quad j = 2, 3, \ldots, n,$$

to eliminate the variable x_1 from the remaining equations. This process is then repeated sequentially for $i = 2, 3, \ldots, n-1$, using

$$\mathrm{R}_j \mapsto \mathrm{R}_j - \frac{a_{ji}}{a_{ii}} \mathrm{R}_i, \quad j = i+1, i+2, \ldots, n.$$

The resulting augmented matrix has the form

$$\tilde{\tilde{\mathbf{A}}} = [\mathbf{A}, \mathbf{b}] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \ldots & a_{1n} & a_{1,n+1} \\ 0 & \tilde{a}_{22} & \ldots & \tilde{a}_{2n} & \tilde{a}_{2,n+1} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \ldots & \tilde{a}_{nn} & \tilde{a}_{n,n+1} \end{array}\right].$$

This system has the same solution set as Eq. (3.1), but it is now in upper triangular form. The corresponding equations are

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= a_{1,n+1}, \\ \tilde{a}_{22}x_2 + \cdots + \tilde{a}_{2n}x_n &= \tilde{a}_{2,n+1}, \\ \vdots \quad & \quad \vdots \\ \tilde{a}_{nn}x_n &= \tilde{a}_{n,n+1}. \end{aligned}$$

From the last equation we obtain

$$x_n = \frac{\tilde{a}_{n,n+1}}{\tilde{a}_{nn}}.$$

Proceeding upward, we compute

$$x_i = \frac{\tilde{a}_{i,n+1} - \sum_{j=i+1}^{n} \tilde{a}_{ij}x_j}{\tilde{a}_{ii}}, \quad i = n-1, n-2, \ldots, 1.$$

Gaussian elimination can also be viewed as a sequence of augmented matrices $\tilde{\mathbf{A}}^{(1)}, \tilde{\mathbf{A}}^{(2)}, \ldots, \tilde{\mathbf{A}}^{(n)}$, where $\tilde{\mathbf{A}}^{(1)}$ is defined in Eq. (3.3). A general matrix $\tilde{\mathbf{A}}^{(k)}$, for $k = 2, 3, \ldots, n$, has the form

$$\tilde{\mathbf{A}}^{(k)} = \left[\begin{array}{cccc|c} a_{11}^{(1)} & \ldots & a_{1n}^{(1)} & & a_{1,n+1}^{(1)} \\ 0 & a_{22}^{(2)} & \ldots & a_{2n}^{(2)} & a_{2,n+1}^{(2)} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \ldots & a_{nn}^{(k)} & a_{n,n+1}^{(k)} \end{array}\right], \tag{3.4}$$

where the variable x_{k-1} has been eliminated from rows $\mathrm{R}_k, \ldots, \mathrm{R}_n$.

In general, the matrix entries are given by

$$a_{ij}^{(k)} = \begin{cases} a_{ij}^{(k-1)}, & i < k, \\ 0, & i \geq k,\ j < k, \\ a_{ij}^{(k-1)} - \dfrac{a_{i,k-1}^{(k-1)}}{a_{k-1,k-1}^{(k-1)}} a_{k-1,j}^{(k-1)}, & i \geq k,\ j \geq k. \end{cases}$$

Example 3.1. Consider the system of linear equations represented by the augmented matrix

$$[\mathbf{A}, \mathbf{b}] = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & 3 & 1 & 11 \\ 3 & 2 & 2 & 13 \end{array}\right].$$

Using the row operations $\mathrm{R}_2 \mapsto \mathrm{R}_2 - 2\mathrm{R}_1$ and $\mathrm{R}_3 \mapsto \mathrm{R}_3 - 3\mathrm{R}_1$, we obtain

$$[\mathbf{A}, \mathbf{b}] = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -1 & -1 \\ 0 & -1 & -1 & -5 \end{array}\right].$$

Next, applying the row operation $\mathrm{R}_3 \mapsto \mathrm{R}_3 + \mathrm{R}_2$ yields

$$[\mathbf{A}, \mathbf{b}] = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -1 & -1 \\ 0 & 0 & -2 & -6 \end{array}\right].$$

The resulting system is upper triangular. Using backward substitution, we obtain

$$\mathbf{x} = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}^\top.$$

This procedure will fail if any of the diagonal elements $\{a_{ii}^{(i)}\}$ for $i = 1, 2, \ldots, n$ is zero, since the row operation

$$\mathrm{R}_i \mapsto \mathrm{R}_i - \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}} \mathrm{R}_k$$

cannot be performed. In such cases, backward substitution may also fail.

The system may still possess a solution, but the elimination technique must be suitably modified.

Example 3.2. Consider the augmented matrix

$$\tilde{\mathbf{A}} = \tilde{\mathbf{A}}^{(1)} = \left[\begin{array}{rrrr|r} 1 & -1 & 2 & -1 & -8 \\ 2 & -2 & 3 & -3 & -20 \\ 1 & 1 & 1 & 0 & -2 \\ 1 & -1 & 4 & 3 & 4 \end{array}\right].$$

Performing the row operations $\mathrm{R}_3 \mapsto \mathrm{R}_3 - \mathrm{R}_1$, $\mathrm{R}_4 \mapsto \mathrm{R}_4 - \mathrm{R}_1$, and $\mathrm{R}_2 \mapsto \mathrm{R}_2 - 2\mathrm{R}_1$, we obtain

$$\tilde{\mathbf{A}} = \tilde{\mathbf{A}}^{(2)} = \left[\begin{array}{rrrr|r} 1 & -1 & 2 & -1 & -8 \\ 0 & 0 & -1 & -1 & -4 \\ 0 & 2 & -1 & 1 & 6 \\ 0 & 0 & 2 & 4 & 12 \end{array}\right].$$

Here, the element $a_{22}^{(2)} = 0$ is called the *pivot*. Since the pivot is zero, the elimination procedure cannot proceed in its current form. We therefore search the second column for the first non-zero entry below the second row. Since $a_{32}^{(2)} \neq 0$, we perform the row interchange $\mathrm{R}_2 \leftrightarrow \mathrm{R}_3$ and then continue with the elimination process.

The above example illustrates the situation when $a_{kk}^{(k)} = 0$ for some $k = 1, 2, \ldots, n-1$. In this case, the following procedure is adopted:

1. The kth column of $\tilde{\mathbf{A}}^{(k-1)}$ is searched from the kth row to the nth row for the first non-zero entry $a_{pk}^{(k)} \neq 0$, where $k \leq p \leq n$.

2. The row interchange $\mathrm{R}_p \leftrightarrow \mathrm{R}_k$ is performed to obtain a temporary matrix, after which the standard elimination process is continued.

If $a_{pk}^{(k)} = 0$ for all $p = k, k+1, \ldots, n$, then the system does not possess a unique solution, since the columns of the coefficient matrix are linearly dependent[2]. Finally, if $a_{nn}^{(n)} = 0$, the system again fails to have a unique solution.

The algorithm for Gaussian elimination is presented in Algorithm 4. Although the algorithm appears to generate new matrices $\tilde{\mathbf{A}}^{(i)}$ for $i = 1, 2, \ldots, n$, in practice all computations can be performed using a single $n \times (n+1)$ augmented matrix.

Computational Complexity

We now discuss the computational complexity of Gaussian elimination. In numerical computations, the time required to perform a multiplication or a division is generally greater than that required for an addition or a subtraction. Rather than deriving the exact operation counts in detail, we simply state the results, which can be verified by a direct counting of arithmetic operations.

[2]Two vectors $\mathbf{x}_1$ and $\mathbf{x}_2$ are said to be linearly dependent if there exist scalars c_1, c_2, not both zero, such that

$$c_1\mathbf{x}_1 + c_2\mathbf{x}_2 = \mathbf{0}.$$

Algorithm 4 Gaussian Elimination with Partial Pivoting

Given: Matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, vector $\mathbf{b} \in \mathbb{R}^n$
Find: Solution $\mathbf{x}$ if a unique solution exists
  Step 1: Form the Augmented Matrix
  Construct $\tilde{\mathbf{A}} = [\mathbf{A} \mid \mathbf{b}]$
  Step 2: Forward Elimination
  for $i = 1$ **to** $n - 1$ **do**
    Step 2.1: Pivot Selection
    Find $p \in \{i, \ldots, n\}$ such that $\tilde{\mathbf{A}}_{pi} \neq 0$
    if no such p exists **then**
      Output("No unique solution")
      exit()
    end if
    Step 2.2: Row Exchange
    if $p \neq i$ **then**
      Exchange rows $\mathrm{R}_i \leftrightarrow \mathrm{R}_p$
    end if
    Step 2.3: Elimination Below Pivot
    for $k = i + 1$ **to** n **do**
      $m_{ki} = \dfrac{\tilde{\mathbf{A}}_{ki}}{\tilde{\mathbf{A}}_{ii}}$
      for $j = i$ **to** $n + 1$ **do**
        $\tilde{\mathbf{A}}_{kj} = \tilde{\mathbf{A}}_{kj} - m_{ki}\tilde{\mathbf{A}}_{ij}$
      end for
    end for
  end for
  Step 3: Check Last Pivot
  if $\tilde{\mathbf{A}}_{nn} = 0$ **then**
    Output("No unique solution")
    exit()
  end if
  Step 4: Backward Substitution
  Initialize $\mathbf{x} \in \mathbb{R}^n$
  $x_n = \dfrac{\tilde{\mathbf{A}}_{n,n+1}}{\tilde{\mathbf{A}}_{nn}}$
  for $i = n - 1$ **to** 1 **do**
    $\mathrm{sum} = 0$
    for $j = i + 1$ **to** n **do**
      $\mathrm{sum} = \mathrm{sum} + \tilde{\mathbf{A}}_{ij}x_j$
    end for
    $x_i = \dfrac{\tilde{\mathbf{A}}_{i,n+1} - \mathrm{sum}}{\tilde{\mathbf{A}}_{ii}}$
  end for
  return $\mathbf{x}$

Gaussian elimination can be divided into two main stages:

1. **Row-Reduction:** In this stage, the augmented matrix is reduced to an upper triangular matrix. The total number of additions and subtractions required is

$$\frac{n^3 - n}{3},$$

while the total number of multiplications and divisions required is

$$\frac{2n^3 + 3n^2 - 5n}{6}.$$

2. **Backward Substitution:** In this stage, the total number of additions and subtractions required is

$$\frac{n^2 - n}{2},$$

and the total number of multiplications and divisions required is

$$\frac{n^2 + n}{2}.$$

Therefore, the total number of additions and subtractions required by Gaussian elimination is

$$\frac{n^3}{3} + n^2 - \frac{n}{3},$$

while the total number of multiplications and divisions required is

$$\frac{n^3}{3} + \frac{n^2}{2} - \frac{5}{6}n.$$

Thus, the total number of arithmetic operations grows proportionally to n^3. Consequently, Gaussian elimination is said to have computational complexity $\mathcal{O}(n^3)$.

3.1.2 Gauss–Jordan Algorithm

Wilhelm Jordan was a geodesist who extended the classical Gaussian elimination procedure to obtain a matrix in *reduced row-echelon form*. He should not be confused with Camille Jordan, who is known for the Jordan curve theorem and the Jordan canonical form.

The Gauss–Jordan method is a variation of Gaussian elimination in which the variable x_i is eliminated not only from the rows $\mathrm{R}_{i+1}, \mathrm{R}_{i+2}, \ldots, \mathrm{R}_n$, but also from the rows $\mathrm{R}_1, \mathrm{R}_2, \ldots, \mathrm{R}_{i-1}$. After completing this process for all variables, the augmented matrix takes the diagonal form

$$[\mathbf{A}, \mathbf{b}] = \left[\begin{array}{cccc|c} a_{11}^{(1)} & 0 & \ldots & 0 & a_{1,n+1}^{(1)} \\ 0 & a_{22}^{(2)} & \ldots & 0 & a_{2,n+1}^{(2)} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \ldots & a_{nn}^{(n)} & a_{n,n+1}^{(n)} \end{array}\right].$$

Figure 3.3: Wilhelm Jordan (1 March 1842 – 17 April 1899).

The solution of the system can then be obtained directly as

$$x_i = \frac{a_{i,n+1}^{(i)}}{a_{ii}^{(i)}}, \qquad i = 1, 2, \ldots, n.$$

The Gauss–Jordan algorithm is summarised in Algorithm 5.

3.2 Matrix Factorisation

Gaussian elimination consists of two steps: the row-reduction step and backward substitution. The former has a computational complexity of $\mathcal{O}(n^3)$, while the latter requires only $\mathcal{O}(n^2)$. This means that if we have a triangular matrix, solving the system requires only $\mathcal{O}(n^2)$ operations.

3.2.1 LU Decomposition

Suppose that we have $\mathbf{A} = \mathbf{LU}$, meaning that $\mathbf{A}$ has been factored into a lower triangular matrix[3] ($\mathbf{L}$) and an upper triangular matrix[4] ($\mathbf{U}$). Then, solving $\mathbf{Ax} = \mathbf{b}$ can be done in two steps:

- Solve $\mathbf{Ly} = \mathbf{b}$ for $\mathbf{y}$.
- Solve $\mathbf{Ux} = \mathbf{y}$ for $\mathbf{x}$.

Both steps require only $\mathcal{O}(n^2)$ operations.

Although LU decomposition was developed by several mathematicians, the Polish mathematician Tadeusz Banachiewicz is credited with important contributions to its systematic formulation.

LU decomposition reduces the cost of solving a linear system from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$ per right-hand-side vector. However, the factorization of $\mathbf{A}$ into $\mathbf{L}$ and $\mathbf{U}$ itself requires $\mathcal{O}(n^3)$ operations. Once computed, this factorization can be stored and reused for multiple right-hand-side vectors $\mathbf{b}$.

[3]**Lower Triangular Matrix:** A matrix $\mathbf{A} = \{a_{ij}\}_{i,j=1}^n$ is said to be lower triangular if $a_{ij} = 0$ for $i < j$.
[4]**Upper Triangular Matrix:** A matrix $\mathbf{A} = \{a_{ij}\}_{i,j=1}^n$ is said to be upper triangular if $a_{ij} = 0$ for $j < i$.

Algorithm 5 Gauss–Jordan Elimination

Given: Matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, vector $\mathbf{b} \in \mathbb{R}^n$
Find: Solution $\mathbf{x}$ if a unique solution exists
  Step 1: Form the Augmented Matrix
  Construct $\tilde{\mathbf{A}} = [\mathbf{A} \mid \mathbf{b}]$
  Step 2: Reduce to Reduced Row-Echelon Form
  for $i = 1$ **to** n **do**
    Step 2.1: Pivot Selection
    Find $p \in \{i, \ldots, n\}$ such that $\tilde{\mathbf{A}}_{pi} \neq 0$
    if no such p exists **then**
      Output("No unique solution")
      exit()
    end if
    Step 2.2: Row Exchange
    if $p \neq i$ **then**
      Exchange rows $\mathrm{R}_i \leftrightarrow \mathrm{R}_p$
    end if
    Step 2.3: Normalize Pivot Row
    Divide row R_i by $\tilde{\mathbf{A}}_{ii}$
    Step 2.4: Eliminate Other Entries in Column i
    for $k = 1$ **to** n **do**
      if $k \neq i$ **then**
        $m_{ki} = \tilde{\mathbf{A}}_{ki}$
        for $j = i$ **to** $n + 1$ **do**
          $\tilde{\mathbf{A}}_{kj} = \tilde{\mathbf{A}}_{kj} - m_{ki}\tilde{\mathbf{A}}_{ij}$
        end for
      end if
    end for
  end for
  Step 3: Extract Solution
  Initialize $\mathbf{x} \in \mathbb{R}^n$
  for $i = 1$ **to** n **do**
    $x_i = \tilde{\mathbf{A}}_{i,n+1}$
  end for
  return $\mathbf{x}$

Figure 3.4: Tadeusz Banachiewicz (13 February 1882 – 17 November 1954).

To proceed with LU decomposition, we assume that the system $\mathbf{Ax} = \mathbf{b}$ can be solved using Gaussian elimination without row pivoting, i.e.,

$$a_{ii}^{(i)} \neq 0, \qquad i = 1, 2, \ldots, n.$$

The first step in Gaussian elimination consists of performing, for each $j = 2, 3, \ldots, n$,

$$\mathrm{R}_j \mapsto \mathrm{R}_j - m_{j1}\mathrm{R}_1, \qquad \text{where} \quad m_{j1} = \frac{a_{j1}^{(1)}}{a_{11}^{(1)}}.$$

An equivalent interpretation is obtained by multiplying $\mathbf{A}$ on the left by the matrix $\mathbf{M}^{(1)}$, where

$$\mathbf{M}^{(1)} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ -m_{21} & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ -m_{n1} & 0 & \cdots & 1 \end{bmatrix}.$$

This matrix is called the *first Gaussian transformation matrix.* The resulting matrix is denoted by $\mathbf{A}^{(2)}$, where

$$\mathbf{A}^{(2)} = \mathbf{M}^{(1)}\mathbf{A},$$

and the right-hand-side vector is updated as

$$\mathbf{b}^{(2)} = \mathbf{M}^{(1)}\mathbf{b}.$$

Next, we construct $\mathbf{M}^{(2)}$ by replacing the subdiagonal entries in the second column of the identity matrix with the negative of the multipliers

$$m_{j2} = \frac{a_{j2}^{(2)}}{a_{22}^{(2)}}.$$

This process continues until an upper triangular matrix $\mathbf{A}^{(n)}$ is obtained:

$$\mathbf{A}^{(n)} = \mathbf{M}^{(n-1)}\mathbf{M}^{(n-2)} \cdots \mathbf{M}^{(1)}\mathbf{A}.$$

We define

$$\mathbf{U} = \mathbf{A}^{(n)}.$$

To compute the lower triangular matrix $\mathbf{L}$, observe that the inverse of each Gaussian transformation matrix $\mathbf{M}^{(k)}$ is given by

$$\mathbf{L}^{(k)} = \left[\mathbf{M}^{(k)}\right]^{-1} = \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & m_{k+1,k} & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & m_{n,k} & 0 & \cdots & 1 \end{bmatrix}.$$

The lower triangular matrix $\mathbf{L}$ is then defined as

$$\mathbf{L} = \mathbf{L}^{(1)}\mathbf{L}^{(2)} \cdots \mathbf{L}^{(n-1)}.$$

Since $\mathbf{L}^{(k)} = \left[\mathbf{M}^{(k)}\right]^{-1}$, we conclude that

$$\mathbf{LU} = \mathbf{A}.$$

Theorem 3.1. (Doolittle LU Decomposition) *If Gaussian elimination can be performed on the system* $\mathbf{Ax} = \mathbf{b}$ *without row interchanges, then the matrix* $\mathbf{A}$ *can be factored as* $\mathbf{A} = \mathbf{LU}$*, where*

$$m_{ji} = \frac{a_{ji}^{(i)}}{a_{ii}^{(i)}},$$

and

$$\mathbf{U} = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} \\ 0 & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn}^{(n)} \end{bmatrix}, \quad \mathbf{L} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ m_{21} & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ m_{n1} & m_{n2} & \cdots & 1 \end{bmatrix}.$$

The above factorization is known as the Doolittle method, where the matrix $\mathbf{L}$ has ones on its diagonal. Alternatively, if the diagonal entries of $\mathbf{U}$ are equal to one, the resulting factorization is called Crout's LU decomposition. The algorithm of LU decomposition method is given in Algorithm 6.

Algorithm 6 LU Decomposition

Given: Matrix $\mathbf{A}$ of size $n \times n$
Find: Matrices $\mathbf{L}$ and $\mathbf{U}$ such that $\mathbf{A} = \mathbf{LU}$
  Step 1: Initialize Matrices
  Initialize $\mathbf{L}$ as an $n \times n$ identity matrix
  Initialize $\mathbf{U}$ as $\mathbf{A}$
  Step 2: Perform LU Decomposition
  for $i = 1$ **to** $n - 1$ **do**
    Step 2.1: Check Pivot
    if $\mathbf{U}_{ii} = 0$ **then**
      Output("Zero pivot encountered. LU decomposition without pivoting fails.")
      exit()
    end if
    Step 2.2: Matrix Reduction
    for $k = i + 1$ **to** n **do**
      $m_{ki} = \dfrac{\mathbf{U}_{ki}}{\mathbf{U}_{ii}}$
      $\mathbf{L}_{ki} = m_{ki}$
      for $j = i$ **to** n **do**
        $\mathbf{U}_{kj} = \mathbf{U}_{kj} - m_{ki}\mathbf{U}_{ij}$
      end for
    end for
  end for
  return $\mathbf{L}, \mathbf{U}$

Example 3.3. In hand computations, we often compute the LU factorisation in a slightly different way. Consider the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 1 \\ 3 & 2 & 2 \end{bmatrix}.$$

Let

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{bmatrix}, \qquad \mathbf{U} = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}.$$

Then

$$\mathbf{LU} = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ \ell_{21}u_{11} & \ell_{21}u_{12} + u_{22} & \ell_{21}u_{13} + u_{23} \\ \ell_{31}u_{11} & \ell_{31}u_{12} + \ell_{32}u_{22} & \ell_{31}u_{13} + \ell_{32}u_{23} + u_{33} \end{bmatrix}.$$

Equating $\mathbf{LU} = \mathbf{A}$ and comparing coefficients, we obtain

$$\ell_{21} = 2, \qquad \ell_{31} = 3, \qquad \ell_{32} = -1,$$

and

$$u_{11} = 1, \quad u_{12} = 1, \quad u_{13} = 1, \quad u_{22} = 1, \quad u_{23} = -1, \quad u_{33} = -2.$$

Thus,

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & -1 & 1 \end{bmatrix}, \quad 55 \quad \mathbf{U} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & -2 \end{bmatrix}.$$

Once the LU factorization is obtained, the system $\mathbf{Ax} = \mathbf{b}$ is solved efficiently by first computing $\mathbf{y}$ from $\mathbf{Ly} = \mathbf{b}$ using forward substitution, followed by solving $\mathbf{Ux} = \mathbf{y}$ using backward substitution.

It is important to note that not all square matrices admit an LU factorization. For example, the matrix

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

has no LU factorization without row permutations.

Finally, we remark that the LU decomposition is not unique.

Theorem 3.2. *If a matrix has an LU decomposition, then it is not unique.*

Proof. Let $\mathbf{A}$ admit an LU decomposition, i.e., $\mathbf{A} = \mathbf{LU}$. Then

$$\begin{aligned} \mathbf{A} &= \mathbf{LU} \\ &= \mathbf{LDD}^{-1}\mathbf{U} \\ &= (\mathbf{LD})(\mathbf{D}^{-1}\mathbf{U}), \end{aligned}$$

where $\mathbf{D}$ is any nonsingular diagonal matrix. Since $\mathbf{LD}$ is lower triangular and $\mathbf{D}^{-1}\mathbf{U}$ is upper triangular, this produces infinitely many LU decompositions of $\mathbf{A}$. □

Remark 3.1. So far, we have assumed that LU decomposition is applicable to systems of equations that do not require pivoting. However, in general, pivoting is necessary. This is referred to as **PLU** decomposition. Interested readers can refer to [3, Section 6.5].

3.2.2 LDL$^\top$ & Cholesky Decomposition

In linear algebra, certain special matrices possess useful structural properties. These properties also reflect in their matrix factorizations, particularly in LU-type decompositions. We first introduce some important classes of matrices, discuss their properties, and then describe their special factorizations.

Definition 3.1 (Diagonally Dominant Matrix). A matrix $\mathbf{A} = (a_{ij})$ is said to be *diagonally dominant* if

$$|a_{ii}| \geq \sum_{\substack{j=1 \\ j \neq i}}^{n} |a_{ij}|, \qquad i = 1, 2, \ldots, n.$$

If the inequality is strict for every i, then $\mathbf{A}$ is called *strictly diagonally dominant*.

Example 3.4. Consider the matrices

$$\mathbf{A}_1 = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 3 & 2 \\ 1 & 2 & 3 \end{bmatrix}, \quad \mathbf{A}_2 = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 3 & 2 \\ 1 & 2 & 1 \end{bmatrix}, \quad \mathbf{A}_3 = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 3 & 2 \\ 1 & 2 & 5 \end{bmatrix}.$$

Then $\mathbf{A}_1$ is diagonally dominant, $\mathbf{A}_2$ is not diagonally dominant, and $\mathbf{A}_3$ is strictly diagonally dominant.

One of the key important properties of strictly diagonally dominant matrices is that they are nonsingular.

Theorem 3.3. *A strictly diagonally dominant matrix* $\mathbf{A}$ *is nonsingular. Moreover, Gaussian elimination applied to a system* $\mathbf{Ax} = \mathbf{b}$ *can be carried out without row or column interchanges, and the computation is stable with respect to the growth of round-off errors.*

The above theorem also implies that *pivoting is not required* for strictly diagonally dominant matrices.

Next, we look at another important type of matrices, where we can further simplify the **LU** decomposition.

Definition 3.2 (**Positive Definite Matrix**)**.** A matrix $\mathbf{A}$ is said to be *positive definite* if

$$\mathbf{x}^\top \mathbf{A}\mathbf{x} > 0 \quad \text{for all } \mathbf{x} \neq \mathbf{0}.$$

If $\mathbf{A}$ is symmetric and positive definite, it is called a *symmetric positive definite (SPD)* matrix.

Example 3.5. Consider the matrices

$$\mathbf{A}_1 = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}, \qquad \mathbf{A}_2 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

For an arbitrary vector $\mathbf{x} = (x_1, x_2)^\top \in \mathbb{R}^2$,

$$\mathbf{x}^\top \mathbf{A}_1 \mathbf{x} = x_1^2 + x_2^2 + (x_1 - x_2)^2 > 0,$$

and hence $\mathbf{A}_1$ is positive definite.
For $\mathbf{A}_2$,

$$\mathbf{x}^\top \mathbf{A}_2 \mathbf{x} = x_1^2 - x_2^2.$$

Taking $x_1 = 0$ and $x_2 \neq 0$, we obtain $\mathbf{x}^\top \mathbf{A}_2 \mathbf{x} < 0$, and therefore $\mathbf{A}_2$ is not positive definite.

For the next few theorems and corollaries, proofs are omitted and can be found in standard linear algebra texts; see, for example, [6].

To state a necessary and sufficient condition for symmetric positive definiteness, we introduce the notion of a leading principal submatrix.

Definition 3.3 (Leading Principal Submatrix). A *leading principal submatrix* of a matrix $\mathbf{A}$ is a matrix of the form

$$\mathbf{A}_k = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1k} \\ a_{21} & a_{22} & \cdots & a_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{k1} & a_{k2} & \cdots & a_{kk} \end{bmatrix},$$

for some $k = 1, 2, \ldots, n$.

Theorem 3.4 (Necessary and Sufficient Condition for SPD). *A symmetric matrix $\mathbf{A}$ is symmetric positive definite if and only if all its leading principal submatrices have positive determinants.*

Example 3.6. Consider the matrix

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}.$$

Its leading principal submatrices $\mathbf{A}_1$, $\mathbf{A}_2$, and $\mathbf{A}_3$ have determinants 2, 3, and 4, respectively, all of which are positive. Hence, $\mathbf{A}$ is symmetric positive definite.

Theorem 3.5. *A symmetric matrix $\mathbf{A}$ is symmetric positive definite if and only if Gaussian elimination without row interchanges can be applied to the linear system $\mathbf{Ax} = \mathbf{b}$ with all pivot elements positive. Moreover, in this case, the computations are stable with respect to the growth of round-off errors.*

We are not interested in the proof of the above theorem, but rather in the important corollaries that arise from it.

Corollary 3.6 ($\mathbf{LDL}^\top$ Factorisation). *A matrix $\mathbf{A}$ is symmetric positive definite if and only if it admits a factorisation of the form $\mathbf{A} = \mathbf{LDL}^\top$, where $\mathbf{L}$ is a unit lower triangular matrix and $\mathbf{D}$ is a diagonal matrix with positive diagonal entries.*

Corollary 3.6 has a counterpart when $\mathbf{A}$ is symmetric but not necessarily positive definite.

Corollary 3.7. *Let* $\mathbf{A}$ *be a symmetric matrix for which Gaussian elimination can be applied without row interchanges. Then* $\mathbf{A}$ *admits a factorisation of the form* $\mathbf{A} = \mathbf{LDL}^\top$*, where* $\mathbf{L}$ *is a unit lower triangular matrix and* $\mathbf{D}$ *is a diagonal matrix whose diagonal entries are the pivots* $a_{11}^{(1)}, a_{22}^{(2)}, \ldots, a_{nn}^{(n)}$*.*

The algorithm for computing the $\mathbf{LDL}^\top$ factorisation is presented in Algorithm 7.

Algorithm 7 LDL$^\top$ Decomposition

Given: Symmetric matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$
Find: Lower triangular matrix $\mathbf{L}$ with unit diagonal and diagonal matrix $\mathbf{D}$ such that $\mathbf{A} = \mathbf{LDL}^T$
  Step 1: Initialize Matrices
  Initialize $\mathbf{L}$ as the $n \times n$ identity matrix
  Initialize $\mathbf{D}$ as a zero matrix of size $n \times n$
  Step 2: Compute D and L
  for $i = 1$ **to** n **do**
    Step 2.1: Compute Diagonal Entry $\mathbf{D}_{ii}$
    sum $= 0$
    for $k = 1$ **to** $i - 1$ **do**
      sum $=$ sum $+ \mathbf{D}_{kk}\mathbf{L}_{ik}^2$
    end for
    $\mathbf{D}_{ii} = \mathbf{A}_{ii} -$ sum
    if $\mathbf{D}_{ii} = 0$ **then**
      Output("Zero pivot encountered. LDL$^\top$ decomposition fails.")
      exit()
    end if
    Step 2.2: Compute Subdiagonal Entries of L in Column i
    for $j = i + 1$ **to** n **do**
      sum $= 0$
      for $k = 1$ **to** $i - 1$ **do**
        sum $=$ sum $+ \mathbf{D}_{kk}\mathbf{L}_{ik}\mathbf{L}_{jk}$
      end for
      $\mathbf{L}_{ji} = \dfrac{\mathbf{A}_{ji} - \text{sum}}{\mathbf{D}_{ii}}$
    end for
  end for
  return $\mathbf{L}, \mathbf{D}$

Algorithm 7 is based on computing the individual entries of the matrices $\mathbf{L}$ and $\mathbf{D}$. We illustrate how these entries arise by working through a small example, which also clarifies how the algorithm is constructed.

Example 3.7. Let $\mathbf{A}$ be a 3×3 symmetric matrix that admits an $\mathrm{LDL}^\top$ decomposition. Then

$$\begin{aligned}\mathbf{A} &= \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{bmatrix} \begin{bmatrix} d_{11} & 0 & 0 \\ 0 & d_{22} & 0 \\ 0 & 0 & d_{33} \end{bmatrix} \begin{bmatrix} 1 & \ell_{21} & \ell_{31} \\ 0 & 1 & \ell_{32} \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} d_{11} & d_{11}\ell_{21} & d_{11}\ell_{31} \\ d_{11}\ell_{21} & d_{22} + d_{11}\ell_{21}^2 & d_{22}\ell_{32} + d_{11}\ell_{21}\ell_{31} \\ d_{11}\ell_{31} & d_{22}\ell_{32} + d_{11}\ell_{21}\ell_{31} & d_{11}\ell_{31}^2 + d_{22}\ell_{32}^2 + d_{33} \end{bmatrix}.\end{aligned}$$

From this expression we immediately obtain

$$d_{11} = a_{11}, \qquad \ell_{i1} = \frac{a_{i1}}{d_{11}}, \quad i = 2, 3.$$

Once d_{11} and ℓ_{21}, ℓ_{31} are known, we can compute d_{22}, followed by ℓ_{32}, and finally d_{33}. The same procedure extends naturally to an $n \times n$ matrix.

Cholesky Decomposition

From Theorem 3.5, we obtain another important corollary for symmetric positive definite matrices, leading to a simpler factorisation.

Corollary 3.8 (Cholesky Decomposition). *A matrix* $\mathbf{A}$ *is symmetric positive definite if and only if it can be factored in the form*

$$\mathbf{A} = \mathbf{L}\mathbf{L}^\top,$$

where $\mathbf{L}$ *is a lower triangular matrix with strictly positive diagonal entries.*

The Cholesky decomposition was discovered by André-Louis Cholesky, a French military officer and mathematician.

Figure 3.5: André-Louis Cholesky (15 October 1875 – 31 August 1918).

The algorithm for computing the Cholesky decomposition is given in Algorithm 8.

Algorithm 8 is based on the computation of individual entries of the matrix $\mathbf{L}$. We illustrate how these entries arise through a simple example.

Algorithm 8 Cholesky Decomposition

Given: Symmetric positive definite matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$
Find: Lower triangular matrix $\mathbf{L}$ such that $\mathbf{A} = \mathbf{L}\mathbf{L}^T$

```
Step 1: Initialize Matrix
Initialize L as a zero matrix of size n × n
Step 2: Compute L
for i = 1 to n do
  Step 2.1: Compute Diagonal Entry L_ii
  sum = 0
  for k = 1 to i − 1 do
    sum = sum + L_ik^2
  end for
  if A_ii − sum ≤ 0 then
    Output("Matrix is not positive definite")
    exit()
  end if
  L_ii = √(A_ii − sum)
  Step 2.2: Compute Subdiagonal Entries in Column i
  for j = i + 1 to n do
    sum = 0
    for k = 1 to i − 1 do
      sum = sum + L_jk L_ik
    end for
    L_ji = (A_ji − sum) / L_ii
  end for
end for
return L
```

Example 3.8. Let $\mathbf{A}$ be a 3×3 symmetric positive definite matrix admitting a Cholesky ($\mathrm{LL}^\top$) decomposition. Then

$$\begin{aligned}\mathbf{A} &= \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} \ell_{11} & 0 & 0 \\ \ell_{21} & \ell_{22} & 0 \\ \ell_{31} & \ell_{32} & \ell_{33} \end{bmatrix} \begin{bmatrix} \ell_{11} & \ell_{21} & \ell_{31} \\ 0 & \ell_{22} & \ell_{32} \\ 0 & 0 & \ell_{33} \end{bmatrix} \\ &= \begin{bmatrix} \ell_{11}^2 & \ell_{11}\ell_{21} & \ell_{11}\ell_{31} \\ \ell_{11}\ell_{21} & \ell_{21}^2 + \ell_{22}^2 & \ell_{21}\ell_{31} + \ell_{22}\ell_{32} \\ \ell_{11}\ell_{31} & \ell_{21}\ell_{31} + \ell_{22}\ell_{32} & \ell_{31}^2 + \ell_{32}^2 + \ell_{33}^2 \end{bmatrix}.\end{aligned}$$

From the above expressions, we obtain

$$\ell_{11} = \sqrt{a_{11}}, \qquad \ell_{i1} = \frac{a_{i1}}{\ell_{11}}, \quad i = 2, 3.$$

Once ℓ_{11} and ℓ_{21}, ℓ_{31} are known, we can compute ℓ_{22}, followed by ℓ_{32}, and finally ℓ_{33}. The same procedure extends naturally to an $n \times n$ matrix.

Until now, we have not discussed the computational complexity of the three factorisation methods. Table 3.1 summarises the applicability, computational complexity, and advantages of LU, $\mathrm{LDL}^\top$, and Cholesky factorizations. For brevity, we do not derive the operation counts here; interested students are encouraged to verify them independently.

Method	**Applicability**	**Advantages**	**Computational Complexity**	
			M/D	**A/S**
LU	General square matrices	Works for general matrices but typically requires pivoting	$\frac{n^3}{3} - \frac{n}{3}$	$\frac{n^3}{3} - \frac{n^2}{2} + \frac{n}{6}$
$\mathrm{LDL}^\top$	Symmetric matrices	More stable than LU, reduced storage, avoids pivoting	$\frac{n^3}{6} + n^2 - \frac{7n}{6}$	$\frac{n^3}{6} - \frac{n}{6}$
Cholesky	Symmetric positive definite matrices	Fastest and most efficient, lowest computational cost	$\frac{n^3}{6} + \frac{n^2}{2} - \frac{2n}{3}$	$\frac{n^3}{6} - \frac{n}{6}$

Table 3.1: Applicability, computational complexity, and advantages of LU, $\mathrm{LDL}^\top$, and Cholesky decompositions. M/D: multiplications and divisions; A/S: additions and subtractions.

We observe that the Cholesky decomposition requires the smallest number of arithmetic operations among the three methods. However, this comparison can be slightly misleading, since Cholesky decomposition also requires computing n square roots. Nevertheless, the cost of computing square roots grows only linearly with n and becomes negligible compared to the $\mathcal{O}(n^3)$ arithmetic operations for large n.

3.3 Iterative Methods

Root-finding methods form a class of iterative methods that we have already encountered. In this part of the chapter, we extend these ideas to systems of equations. Before studying iterative methods for systems of equations, we need a way to measure the distance between vectors in $\mathbb{R}^n$. Such a measure allows us to determine whether a sequence of vectors converges to the solution of a system.

Definition 3.4. A *vector norm* on $\mathbb{R}^n$ is a function $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ satisfying the following properties:

1. $\|\mathbf{x}\| \geq 0$ for all $\mathbf{x} \in \mathbb{R}^n$.
2. $\|\mathbf{x}\| = 0$ if and only if $\mathbf{x} = \mathbf{0}$.
3. $\|\alpha\mathbf{x}\| = |\alpha|\,\|\mathbf{x}\|$ for all $\alpha \in \mathbb{R}$ and $\mathbf{x} \in \mathbb{R}^n$.
4. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$.

Definition 3.5. The ℓ_2 and ℓ_∞ norms of a vector $\mathbf{x} = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$ are defined as

$$\|\mathbf{x}\|_2 = \left(\sum_{i=1}^{n} x_i^2\right)^{1/2}, \qquad \|\mathbf{x}\|_\infty = \max_{1\le i\le n} |x_i|.$$

We now define convergence of a sequence of vectors in $\mathbb{R}^n$.

Definition 3.6. A sequence $\{\mathbf{x}^{(k)}\}_{k=1}^{\infty}$ in $\mathbb{R}^n$ is said to converge to $\mathbf{x}$ with respect to a norm $\|\cdot\|$ if for every $\varepsilon > 0$ there exists an integer $N(\varepsilon)$ such that

$$\|\mathbf{x}^{(k)} - \mathbf{x}\| < \varepsilon \quad \text{for all } k \ge N(\varepsilon).$$

Similar to vector norms, we can also define matrix norms. Intuitively, a matrix norm measures how much a matrix can stretch a vector under a given vector norm. The largest such stretching determines the matrix norm.

Theorem 3.9. *If $\|\cdot\|$ is a vector norm on $\mathbb{R}^n$, then*

$$\|\mathbf{A}\| = \max_{\|\mathbf{x}\|=1} \|\mathbf{A}\mathbf{x}\|$$

defines a matrix norm.

Note that we use the same notation $\|\cdot\|$ for both vector and matrix norms; this slight abuse of notation will be clear from the context.

In this chapter, we will mainly consider the ℓ_∞ and ℓ_2 matrix norms, defined by

$$\|\mathbf{A}\|_\infty = \max_{\|\mathbf{x}\|_\infty=1} \|\mathbf{A}\mathbf{x}\|_\infty, \qquad \|\mathbf{A}\|_2 = \max_{\|\mathbf{x}\|_2=1} \|\mathbf{A}\mathbf{x}\|_2.$$

Finally, we give a convenient expression for the ℓ_∞ matrix norm.

Theorem 3.10. *If $\mathbf{A} = (a_{ij})_{i,j=1}^{n}$ is an $n \times n$ matrix, then*

$$\|\mathbf{A}\|_\infty = \max_{1\le i\le n} \sum_{j=1}^{n} |a_{ij}|.$$

Remark 3.2. The matrix norms defined above are induced matrix norms. In some texts, the following quantity is also referred to as the ∞-norm of a matrix:

$$\|\mathbf{A}\|_\infty = \max_{1\le i,j\le n} |a_{ij}|.$$

This is indeed a matrix norm, but it is *not* induced by any vector norm. Another example of a non-induced matrix norm is the Frobenius norm,

$$\|\mathbf{A}\|_{\mathrm{Fr}} = \sqrt{\mathrm{trace}(\mathbf{A}^\top \mathbf{A})}.$$

3.3.1 Jacobi Method

After a brief review of the required linear algebra concepts, we now return to the domain of numerical analysis.

An iterative technique for solving an $n \times n$ linear system $\mathbf{Ax} = \mathbf{b}$ begins with an initial approximation $\mathbf{x}^{(0)}$ to the solution $\mathbf{x}$ and generates a sequence of vectors $\{\mathbf{x}^{(k)}\}_{k=0}^{\infty}$ such that

$$\mathbf{x}^{(k)} \to \mathbf{x} \quad \text{as } k \to \infty.$$

Carl Gustav Jacob Jacobi was a German mathematician who proposed the Jacobi eigenvalue algorithm, an iterative method for computing the eigenvalues and eigenvectors of a real symmetric matrix. The *Jacobi method* studied here is a simplified variant inspired by this idea.

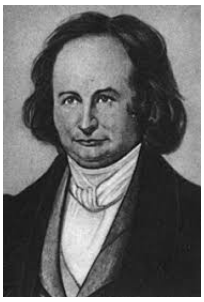

Figure 3.6: Carl Gustav Jacob Jacobi (10 December 1804 – 18 February 1851).

The Jacobi iterative method is obtained by solving the ith equation of the system $\mathbf{Ax} = \mathbf{b}$ for x_i, giving

$$x_i = -\sum_{\substack{j=1 \\ j \neq i}}^{n} \frac{a_{ij}}{a_{ii}} x_j + \frac{b_i}{a_{ii}}, \qquad i = 1, 2, \ldots, n. \tag{3.5}$$

For $k \geq 1$, the components $x_i^{(k)}$ of $\mathbf{x}^{(k)}$ are computed from the components of $\mathbf{x}^{(k-1)}$ as

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^{n} a_{ij} x_j^{(k-1)} \right), \qquad i = 1, 2, \ldots, n. \tag{3.6}$$

Consider the linear system

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n. \end{aligned}$$

Let $\mathbf{x}^{(0)}$ be an initial approximation. The first Jacobi iterate is then given by

$$\begin{aligned}
x_1^{(1)} &= \frac{1}{a_{11}}\left(b_1 - \left(a_{12}x_2^{(0)} + \cdots + a_{1n}x_n^{(0)}\right)\right), \\
x_2^{(1)} &= \frac{1}{a_{22}}\left(b_2 - \left(a_{21}x_1^{(0)} + \cdots + a_{2n}x_n^{(0)}\right)\right), \\
&\vdots \\
x_n^{(1)} &= \frac{1}{a_{nn}}\left(b_n - \left(a_{n1}x_1^{(0)} + \cdots + a_{n,n-1}x_{n-1}^{(0)}\right)\right).
\end{aligned}$$

Subsequent iterates are computed in the same manner for $k \geq 1$.

Example 3.9. Let us take a concrete example. Consider

$$\begin{aligned}
4x_1 + x_2 - x_3 &= 5 \\
-x_1 + 3x_2 + x_3 &= -4 \\
2x_1 + 2x_2 + 5x_3 &= 1.
\end{aligned}$$

As each $a_{ii} \neq 0$, we can rewrite the above system as

$$\begin{aligned}
x_1^{(k)} &= \frac{5}{4} - \frac{1}{4}x_2^{(k-1)} + \frac{1}{4}x_3^{(k-1)} \\
x_2^{(k)} &= -\frac{4}{3} + \frac{1}{3}x_1^{(k-1)} - \frac{1}{3}x_3^{(k-1)} \\
x_3^{(k)} &= \frac{1}{5} - \frac{2}{5}x_1^{(k-1)} - \frac{2}{5}x_2^{(k-1)}.
\end{aligned}$$

Choosing $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^\top$, we get

$$\mathbf{x}^{(1)} = \begin{bmatrix} \frac{5}{4} & -\frac{4}{3} & \frac{1}{5} \end{bmatrix}^\top.$$

Using these values, we get

$$\begin{aligned}
x_1^{(2)} &= \frac{5}{4} + \frac{4}{12} + \frac{1}{20} = 1.633 \\
x_2^{(2)} &= -\frac{4}{3} + \frac{5}{12} - \frac{1}{15} = -0.9833 \\
x_3^{(2)} &= \frac{1}{5} - \frac{1}{2} + \frac{8}{15} = 0.233.
\end{aligned}$$

In general, iterative techniques for solving linear systems rewrite $\mathbf{Ax} = \mathbf{b}$ in the equivalent fixed-point form

$$\mathbf{x} = \mathbf{Tx} + \mathbf{c},$$

for some matrix $\mathbf{T}$ and vector $\mathbf{c}$. Starting from an initial guess $\mathbf{x}^{(0)}$, the iteration is given by

$$\mathbf{x}^{(k)} = \mathbf{Tx}^{(k-1)} + \mathbf{c}, \qquad k = 1, 2, \ldots.$$

For the Jacobi method, the matrix $\mathbf{A}$ is decomposed as

$$\mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U},$$

where $\mathbf{D}$ is the diagonal of $\mathbf{A}$, $\mathbf{L}$ is the strictly lower triangular part, and $\mathbf{U}$ is the strictly upper triangular part. Specifically,

$$\mathbf{D} = \begin{bmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{bmatrix},$$

$$\mathbf{L} = \begin{bmatrix} 0 & 0 & \cdots & 0 \\ -a_{21} & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ -a_{n1} & -a_{n2} & \cdots & 0 \end{bmatrix}, \qquad \mathbf{U} = \begin{bmatrix} 0 & -a_{12} & \cdots & -a_{1n} \\ 0 & 0 & \cdots & -a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}.$$

The system $\mathbf{Ax} = \mathbf{b}$ can then be written as

$$\mathbf{Dx} = (\mathbf{L} + \mathbf{U})\mathbf{x} + \mathbf{b}.$$

Assuming that $\mathbf{D}^{-1}$ exists, we obtain

$$\mathbf{x} = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\mathbf{x} + \mathbf{D}^{-1}\mathbf{b}.$$

Thus, the Jacobi iteration takes the form

$$\mathbf{x}^{(k)} = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\mathbf{x}^{(k-1)} + \mathbf{D}^{-1}\mathbf{b}, \qquad k \geq 1. \tag{3.7}$$

Defining

$$\mathbf{T}_{\mathrm{J}} = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U}), \qquad \mathbf{c}_{\mathrm{J}} = \mathbf{D}^{-1}\mathbf{b},$$

the Jacobi method can be written compactly as

$$\mathbf{x}^{(k)} = \mathbf{T}_{\mathrm{J}}\mathbf{x}^{(k-1)} + \mathbf{c}_{\mathrm{J}}, \qquad k \geq 1. \tag{3.8}$$

For the Jacobi method to be well defined, we require $a_{ii} \neq 0$ for all $i = 1, 2, \ldots, n$. If some diagonal entry $a_{ii} = 0$ but the system is nonsingular, the equations may be reordered so that all diagonal entries are nonzero.

The algorithm for the Jacobi method is presented in Algorithm 9.

3.3.2 Gauss Seidel Method

In the Jacobi method, all components of the vector $\mathbf{x}^{(k-1)}$ are used to compute the components $x_i^{(k)}$ of $\mathbf{x}^{(k)}$. However, for $i > 1$, the components $x_1^{(k)}, \ldots, x_{i-1}^{(k)}$ of $\mathbf{x}^{(k)}$ have already been computed. It is therefore natural to use these newly computed values instead of the older values $x_1^{(k-1)}, \ldots, x_{i-1}^{(k-1)}$, as this is expected to produce a better approximation to the solution.

This idea leads to the Gauss–Seidel method, in which $x_i^{(k)}$ is computed using the most recently available values.

Algorithm 9 Jacobi Iteration Method

Given: Matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ with $a_{ii} \neq 0$, vector $\mathbf{b} \in \mathbb{R}^n$, tolerance ε, maximum iterations N
Find: Approximation $\mathbf{x}$ to the solution of $\mathbf{Ax} = \mathbf{b}$
  Step 1: Initialization
  Initialize $\mathbf{x}^{(0)} = \mathbf{0}$
  Step 2: Jacobi Iteration
  for $k = 1$ **to** N **do**
    for $i = 1$ **to** n **do**
      $\text{sum} = \mathbf{b}_i$
      for $j = 1$ **to** n **do**
        if $j \neq i$ **then**
          $\text{sum} = \text{sum} - a_{ij}x_j^{(k-1)}$
        end if
      end for
      $x_i^{(k)} = \dfrac{\text{sum}}{a_{ii}}$
    end for
    Step 2.1: Check Convergence
    if $\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|_\infty < \varepsilon$ **then**
      return $\mathbf{x}^{(k)}$
    end if
  end for
  Step 3: Maximum Iterations Reached
  Output("Maximum number of iterations reached")
  return $\mathbf{x}^{(N)}$

$$x_i^{(k)} = \frac{1}{a_{ii}}\left[-\sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)} + b_i\right], \qquad i = 1, 2, \ldots, n. \tag{3.9}$$

This iterative scheme is called the *Gauss–Seidel method*. The idea was developed by Carl Friedrich Gauss in the early nineteenth century, it was only published and fully detailed by Seidel in 1874 through a private letter from Gauss to his student Gerling, making the method primarily attributed to both mathematicians.

Figure 3.7: Philipp Ludwig von Seidel (24 October 1821 – 13 August 1896).

Consider a linear system of the form

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n. \end{aligned}$$

Let $\mathbf{x}^{(0)}$ be an initial approximation. The first Gauss–Seidel iterate is then given by

$$\begin{aligned} x_1^{(1)} &= \frac{1}{a_{11}}\left(b_1 - \left(a_{12}x_2^{(0)} + \cdots + a_{1n}x_n^{(0)}\right)\right), \\ x_2^{(1)} &= \frac{1}{a_{22}}\left(b_2 - \left(a_{21}x_1^{(1)} + a_{23}x_3^{(0)} + \cdots + a_{2n}x_n^{(0)}\right)\right), \\ &\vdots \\ x_n^{(1)} &= \frac{1}{a_{nn}}\left(b_n - \left(a_{n1}x_1^{(1)} + a_{n2}x_2^{(1)} + \cdots + a_{n,n-1}x_{n-1}^{(1)}\right)\right). \end{aligned}$$

Subsequent iterates are computed similarly for $k \geq 1$.

Example 3.10. Consider the system

$$\begin{aligned} 4x_1 + x_2 - x_3 &= 5, \\ -x_1 + 3x_2 + x_3 &= -4, \\ 2x_1 + 2x_2 + 5x_3 &= 1. \end{aligned}$$

Since each diagonal entry $a_{ii} \neq 0$, the Gauss–Seidel iteration is

$$\begin{aligned} x_1^{(k)} &= \frac{5}{4} - \frac{1}{4}x_2^{(k-1)} + \frac{1}{4}x_3^{(k-1)}, \\ x_2^{(k)} &= -\frac{4}{3} + \frac{1}{3}x_1^{(k)} - \frac{1}{3}x_3^{(k-1)}, \\ x_3^{(k)} &= \frac{1}{5} - \frac{2}{5}x_1^{(k)} - \frac{2}{5}x_2^{(k)}. \end{aligned}$$

Choosing $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^\top$, we obtain

$$x_1^{(1)} = 1.25, \quad x_2^{(1)} = -0.9166, \quad x_3^{(1)} = -0.0666.$$

Using these values, the second iterate is

$$x_1^{(2)} = 1.4958, \quad x_2^{(2)} = -0.8569, \quad x_3^{(2)} = -0.0556.$$

To write the Gauss–Seidel method in matrix form, we rearrange Eq. (3.9) to obtain

$$(\mathbf{D} - \mathbf{L})\mathbf{x}^{(k)} = \mathbf{U}\mathbf{x}^{(k-1)} + \mathbf{b},$$

where $\mathbf{D}$, $\mathbf{L}$, and $\mathbf{U}$ are defined as in the Jacobi method. Assuming that $(\mathbf{D}-\mathbf{L})$ is nonsingular, we have

$$\mathbf{x}^{(k)} = (\mathbf{D} - \mathbf{L})^{-1}\mathbf{U}\mathbf{x}^{(k-1)} + (\mathbf{D} - \mathbf{L})^{-1}\mathbf{b}, \qquad k \geq 1.$$

Defining

$$\mathbf{T}_{\mathrm{GS}} = (\mathbf{D} - \mathbf{L})^{-1}\mathbf{U}, \qquad \mathbf{c}_{\mathrm{GS}} = (\mathbf{D} - \mathbf{L})^{-1}\mathbf{b},$$

the Gauss–Seidel iteration can be written compactly as

$$\mathbf{x}^{(k)} = \mathbf{T}_{\mathrm{GS}}\mathbf{x}^{(k-1)} + \mathbf{c}_{\mathrm{GS}}.$$

The matrix $\mathbf{D} - \mathbf{L}$ is nonsingular if and only if $a_{ii} \neq 0$ for all $i = 1, 2, \ldots, n$.

In practice, the Gauss–Seidel method often converges faster than the Jacobi method, although this is not guaranteed in all cases.

The algorithm for the Gauss–Seidel method is presented in Algorithm 10.

Algorithm 10 Gauss–Seidel Iteration Method

Given: Matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ with $a_{ii} \neq 0$, vector $\mathbf{b} \in \mathbb{R}^n$, tolerance ε, maximum iterations N
Find: Approximation $\mathbf{x}$ to the solution of $\mathbf{Ax} = \mathbf{b}$
  Step 1: Initialization
  Initialize $\mathbf{x}^{(0)} = \mathbf{0}$
  Step 2: Gauss–Seidel Iteration
  for $k = 1$ **to** N **do**
    for $i = 1$ **to** n **do**
      sum $= \mathbf{b}_i$
      for $j = 1$ **to** n **do**
        if $j < i$ **then**
          sum $=$ sum $- a_{ij}x_j^{(k)}$
        else if $j > i$ **then**
          sum $=$ sum $- a_{ij}x_j^{(k-1)}$
        end if
      end for
      $x_i^{(k)} = \dfrac{\text{sum}}{a_{ii}}$
    end for
    Step 2.1: Check Convergence
    if $\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|_\infty < \varepsilon$ **then**
      return $\mathbf{x}^{(k)}$
    end if
  end for
  Step 3: Maximum Iterations Reached
  Output("Maximum number of iterations reached")
  return $\mathbf{x}^{(N)}$

3.3.3 Error Analysis

We now present some convergence results for the iterative methods discussed in this chapter.

Theorem 3.11 (Convergence of Linear Fixed-Point Iteration). *For any initial vector $\mathbf{x}^{(0)} \in \mathbb{R}^n$, the sequence $\{\mathbf{x}^{(k)}\}_{k=0}^{\infty}$ defined by*

$$\mathbf{x}^{(k)} = \mathbf{T}\mathbf{x}^{(k-1)} + \mathbf{c}, \qquad k \geq 1,$$

converges to the unique fixed point $\mathbf{x} = \mathbf{T}\mathbf{x} + \mathbf{c}$ if and only if

$$\rho(\mathbf{T}) < 1,$$

where $\rho(\mathbf{T})$ denotes the spectral radius of $\mathbf{T}$.

In the above theorem, $\rho(\mathbf{A})$ is called the *spectral radius* of the matrix $\mathbf{A}$.

Definition 3.7 (Spectral Radius). The *spectral radius* of a matrix $\mathbf{A}$ is defined as

$$\rho(\mathbf{A}) = \max_{\lambda \in \sigma(\mathbf{A})} |\lambda|,$$

where $\sigma(\mathbf{A})$ denotes the set of eigenvalues of $\mathbf{A}$.

Based on the above theorem, we obtain an important corollary that provides bounds on the error.

Corollary 3.12 (Error Bounds). *If $\|\mathbf{T}\| < 1$ for some induced (natural) matrix norm and $\mathbf{c}$ is a given vector, then for any $\mathbf{x}^{(0)} \in \mathbb{R}^n$, the sequence $\{\mathbf{x}^{(k)}\}_{k=0}^{\infty}$ defined by*

$$\mathbf{x}^{(k)} = \mathbf{T}\mathbf{x}^{(k-1)} + \mathbf{c}$$

converges to the unique fixed point $\mathbf{x} = \mathbf{T}\mathbf{x} + \mathbf{c}$. Moreover, the following error bounds hold:

1. $$\|\mathbf{x} - \mathbf{x}^{(k)}\| \leq \|\mathbf{T}\|^k \, \|\mathbf{x}^{(0)} - \mathbf{x}\|.$$

2. $$\|\mathbf{x} - \mathbf{x}^{(k)}\| \leq \frac{\|\mathbf{T}\|^k}{1 - \|\mathbf{T}\|} \, \|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\|.$$

Consequently, if

$$\rho(\mathbf{T}_{\mathrm{J}}) < 1 \quad \text{or} \quad \rho(\mathbf{T}_{\mathrm{GS}}) < 1,$$

then the Jacobi or Gauss–Seidel method, respectively, converges.

We now state a sufficient condition that guarantees convergence of both methods.

Theorem 3.13 (Diagonal Dominance). *If the matrix $\mathbf{A}$ is strictly diagonally dominant, then for any initial guess $\mathbf{x}^{(0)} \in \mathbb{R}^n$, both the Jacobi and the Gauss–Seidel methods generate sequences $\{\mathbf{x}^{(k)}\}_{k=0}^{\infty}$ that converge to the unique solution of the system $\mathbf{A}\mathbf{x} = \mathbf{b}$.*

Example 3.11. Consider the system of equations

$$\mathbf{A} = \begin{bmatrix} 4 & 1 & 1 \\ 2 & 7 & 1 \\ 1 & -3 & 12 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 6 \\ 19 \\ 31 \end{bmatrix}.$$

The exact solution of this system is

$$\mathbf{x} = \begin{bmatrix} 0.16346154 & 2.22115385 & 3.125 \end{bmatrix}^\top.$$

Table 3.2 reports the relative error and the number of iterations required by the Jacobi and Gauss–Seidel methods to reach this solution within a prescribed tolerance. We observe that the Gauss–Seidel method requires less than half the number of iterations compared to the Jacobi method, illustrating its faster convergence for this problem.

Gauss–Seidel Method		**Jacobi Method**	
Iteration	Relative Error	Iteration	Relative Error
0	3.0297619047619047	0	2.7142857142857144
1	1.3288690476190474	1	1.3244047619047619
2	1.1076566043083935e-02	2	2.9931972789115680e-01
3	3.1308181444700267e-03	3	6.9745606575963850e-02
4	2.6591374897289020e-04	4	1.6258326247165610e-02
5	1.0467338035224927e-05	5	4.1876931824313960e-03
6	1.0184724974493520e-07	6	1.7016688557863757e-03
7	3.2089560897397010e-08	7	3.5870013378047627e-04
8	2.3501450785445854e-09	8	1.2695183670297094e-04
9	7.9357742599484030e-11	9	4.5606091728322970e-05
		10	1.2516623709157848e-05
		11	4.4680801379870520e-06
		12	1.3864473483726770e-06
		13	4.3891857504085863e-07
		14	1.4488958421932097e-07
		15	4.5451457375556004e-08
		16	1.4724687469325204e-08
		17	4.7262667113301404e-09
		18	1.5090069194201305e-09
		19	4.8637660565731270e-10
		20	1.5566214983664395e-10
		21	4.9958814862804957e-11

Table 3.2: Relative error comparison between Jacobi and Gauss–Seidel methods

In general, there is no universally superior method between Jacobi and Gauss–Seidel. However, under certain additional assumptions, more precise comparisons can be made.

Theorem 3.14. *Suppose that $a_{ij} \leq 0$ for all $i \neq j$ and $a_{ii} > 0$ for $i = 1, 2, \ldots, n$. Then exactly one of the following cases holds:*

1. $$0 \leq \rho(\mathbf{T}_{\mathrm{GS}}) < \rho(\mathbf{T}_{\mathrm{J}}) < 1,$$
2. $$1 < \rho(\mathbf{T}_{\mathrm{J}}) < \rho(\mathbf{T}_{\mathrm{GS}}),$$
3. $$\rho(\mathbf{T}_{\mathrm{J}}) = \rho(\mathbf{T}_{\mathrm{GS}}) = 0,$$
4. $$\rho(\mathbf{T}_{\mathrm{J}}) = \rho(\mathbf{T}_{\mathrm{GS}}) = 1.$$

From the above theorem, we observe that if case (1) holds, then both methods converge, with the Gauss–Seidel method converging faster. If case (2) holds, then both methods diverge, with the Gauss–Seidel method diverging more rapidly.

3.3.4 Condition Number

When solving systems of linear equations, either using *iterative solvers* or *direct solvers*, numerical errors are inevitable. In iterative solvers, errors arise from finite precision arithmetic and truncation, while in direct solvers, round-off errors due to finite precision arithmetic can affect the computed solution.

Consider the system of linear equations

$$\mathbf{A}\mathbf{x} = \mathbf{b},$$

where $\mathbf{A}$ is an invertible matrix. Suppose that a small perturbation $\delta\mathbf{b}$ is introduced in the right-hand side, resulting in a perturbed solution $\mathbf{x} + \delta\mathbf{x}$. The perturbed system can be written as

$$\mathbf{A}(\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b}.$$

Subtracting the original system $\mathbf{A}\mathbf{x} = \mathbf{b}$ gives

$$\mathbf{A}\,\delta\mathbf{x} = \delta\mathbf{b}.$$

Since $\mathbf{A}$ is invertible, we obtain

$$\delta\mathbf{x} = \mathbf{A}^{-1}\delta\mathbf{b}.$$

Taking norms on both sides and using properties of induced matrix norms, we have

$$\|\delta\mathbf{x}\| = \|\mathbf{A}^{-1}\delta\mathbf{b}\| \leq \|\mathbf{A}^{-1}\|\,\|\delta\mathbf{b}\|.$$

Dividing both sides by $\|\mathbf{x}\|$ yields

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq \frac{\|\mathbf{A}^{-1}\|}{\|\mathbf{x}\|}\,\|\delta\mathbf{b}\|.$$

Since $\mathbf{Ax} = \mathbf{b}$, it follows that

$$\|\mathbf{b}\| \leq \|\mathbf{A}\| \, \|\mathbf{x}\|.$$

Using this inequality, we obtain the relative error bound

$$\frac{\|\delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \|\mathbf{A}\| \, \|\mathbf{A}^{-1}\| \frac{\|\delta \mathbf{b}\|}{\|\mathbf{b}\|}.$$

This inequality shows that the relative error in the solution is bounded by the relative error in the right-hand side, scaled by the factor $\|\mathbf{A}\| \, \|\mathbf{A}^{-1}\|$. This motivates the following definition.

Definition 3.8 (Condition Number). The *condition number* of an invertible matrix $\mathbf{A}$ is defined as

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \, \|\mathbf{A}^{-1}\|.$$

The condition number $\kappa(\mathbf{A})$ measures the sensitivity of the solution $\mathbf{x}$ to perturbations in $\mathbf{b}$:

- If $\kappa(\mathbf{A}) \approx 1$, the system is said to be *well-conditioned*.
- If $\kappa(\mathbf{A})$ is large, the system is *ill-conditioned*, and small perturbations in $\mathbf{b}$ may result in large errors in $\mathbf{x}$.

Remark 3.3. The condition number depends on the choice of norm. For example, using the ℓ_∞-induced matrix norm, the condition number is given by

$$\kappa_\infty(\mathbf{A}) = \|\mathbf{A}\|_\infty \, \|\mathbf{A}^{-1}\|_\infty.$$

Example 3.12. Consider the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 1.0001 & 2 \end{bmatrix},$$

whose inverse is

$$\mathbf{A}^{-1} = \begin{bmatrix} -10000 & 10000 \\ 5000.5 & -5000 \end{bmatrix}.$$

Using the ℓ_∞-norm, we compute

$$\|\mathbf{A}\|_\infty = \max\{\, |1| + |2|, \ |1.0001| + |2| \,\} = 3.0001,$$

and

$$\|\mathbf{A}^{-1}\|_\infty = 20000.$$

Hence,

$$\kappa_\infty(\mathbf{A}) = \|\mathbf{A}\|_\infty \, \|\mathbf{A}^{-1}\|_\infty = 60002,$$

which shows that the system is ill-conditioned.

Preconditioners

To reduce the condition number and improve numerical stability, we often employ *preconditioning*. Preconditioning transforms the system $\mathbf{Ax} = \mathbf{b}$ into an equivalent system with improved numerical properties by introducing a matrix $\mathbf{P}$.

Two common types of preconditioning are:

1. **Left preconditioning:**
$$\mathbf{P}^{-1}\mathbf{Ax} = \mathbf{P}^{-1}\mathbf{b}.$$

2. **Right preconditioning:**
$$\mathbf{AP}^{-1}\mathbf{y} = \mathbf{b}, \qquad \text{where } \mathbf{x} = \mathbf{P}^{-1}\mathbf{y}.$$

A good preconditioner $\mathbf{P}$ should satisfy the following properties:

1. The iterative method applied to the preconditioned system converges faster than for the original system.
2. Solving linear systems involving $\mathbf{P}$ is computationally inexpensive.

In practice, a balance must be achieved between these two requirements.

Some commonly used preconditioners include:

1. **Jacobi (diagonal) preconditioner:** $\mathbf{P} = \mathbf{D}$, where $\mathbf{D}$ is the diagonal part of $\mathbf{A}$.
2. **Forward Gauss–Seidel preconditioner:** $\mathbf{P} = \mathbf{D} + \mathbf{L}$.
3. **Backward Gauss–Seidel preconditioner:** $\mathbf{P} = \mathbf{D} + \mathbf{U}$.
4. **Symmetric Gauss–Seidel preconditioner:**
$$\mathbf{P} = (\mathbf{D} + \mathbf{L})\mathbf{D}^{-1}(\mathbf{D} + \mathbf{U}).$$

For convenience, the preconditioner is often described in terms of $\mathbf{P}^{-1}$ rather than $\mathbf{P}$. In practice, iterative methods are almost always used together with preconditioners rather than as standalone solvers.

3.4 Summary

In this chapter, we studied numerical methods for solving systems of linear equations, which play a central role in engineering and scientific computations. The methods discussed can be broadly classified into *direct methods* and *iterative methods*. Each class has its own advantages, limitations, and areas of applicability.

1. **Direct Methods**

- *Gaussian Elimination* transforms a linear system into an equivalent upper triangular system, which can be solved using backward substitution. It is guaranteed to find the exact solution in exact arithmetic, but requires $\mathcal{O}(n^3)$ operations and may suffer from numerical instability without pivoting.
- *LU Decomposition* factorizes the coefficient matrix into lower and upper triangular matrices, allowing efficient solution of systems with multiple right-hand sides.
- *$LDL^\top$ Decomposition* applies to symmetric matrices and reduces storage requirements while improving numerical stability.
- *Cholesky Decomposition* is applicable to symmetric positive definite matrices and is the most efficient among the direct methods discussed, requiring roughly half the computational cost of LU decomposition.

2. **Iterative Methods**
   - *Jacobi Method* computes each iterate using only values from the previous iteration. It is simple to implement and well-suited for parallel computation, but may converge slowly or fail to converge for some systems.
   - *Gauss–Seidel Method* improves upon the Jacobi method by using the most recently computed values within each iteration, typically resulting in faster convergence.
   - The convergence of iterative methods depends on the spectral radius of the iteration matrix. A necessary and sufficient condition for convergence is that the spectral radius be less than one.
   - Strict diagonal dominance of the coefficient matrix provides a sufficient condition for convergence of both Jacobi and Gauss–Seidel methods.
3. **Stability and Conditioning**
   - The *condition number* measures the sensitivity of the solution to perturbations in the data. A large condition number indicates an ill-conditioned system.
   - Preconditioning techniques are used to reduce the condition number and improve convergence rates of iterative solvers.

Choosing a Method:

- For small or dense systems, direct methods such as LU or Cholesky decomposition are preferred.
- For symmetric positive definite systems, Cholesky decomposition is the most efficient choice.
- For large and sparse systems, iterative methods combined with suitable preconditioners are more practical.
- When numerical stability is critical, conditioning and preconditioning must be carefully considered.

Chapter 4

Interpolation

Interpolation has various definitions depending on the source. For example, *Wikipedia* defines interpolation as follows:

"Interpolation is a type of estimation, a method of constructing (finding) new data points within the range of a discrete set of known data points."

Similarly, *Blackphoto* describes interpolation as

"a technique used by digital scanners, cameras, and printers to increase the size of an image in pixels by averaging the colour and brightness values of surrounding pixels."

Such ideas frequently appear in image processing. A rather famous (or infamous) real-world example is the *Ecce Homo* painting (see Fig. 4.1). This fresco, painted in 1930 by the Spanish artist Elías García Martínez, depicts Jesus Christ. Over time, due to wear and tear, the painting deteriorated significantly. In 2012, an 81-year-old woman, Cecilia Giménez, attempted to restore it. As seen in Fig. 4.1, the result was far from satisfactory, and the artwork was humorously renamed *Ecce Mono*. Modern image processing techniques, which inherently rely on interpolation, can achieve significantly better restoration results.

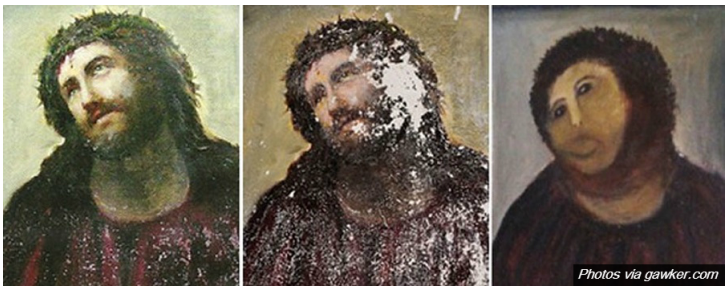

Figure 4.1: Elías García Martínez, *Ecce Homo*. The leftmost photograph (2010) shows early flaking of the paintwork. The central photograph (July 2012) shows the extent of deterioration before restoration. The rightmost photograph shows the artwork after the attempted restoration by Cecilia Giménez.

Another example arises in biological engineering. Suppose we are given population data of a certain species at discrete time intervals, for instance, in the years 1940, 1960, 1970,

and 1980. If we wish to estimate the population in the year 1950, interpolation provides a systematic way to compute this unknown value using the available data.

In interpolation, we aim to approximate a given function by a "simple" class of functions, most commonly polynomials. In analysis, a powerful result that connects continuous functions with polynomial approximation is the Weierstrass Approximation Theorem, proved by Karl Weierstrass.

Figure 4.2: Karl Weierstrass (31 October 1815 – 19 February 1897).

Theorem 4.1 (Weierstrass Approximation Theorem). [1, Theorem 5.4.14] *Let $f \in \mathcal{C}[a,b]$. Then for each $\varepsilon > 0$ there exists a polynomial $p(x)$ such that*

$$|f(x) - p(x)| < \varepsilon \qquad \text{for all } x \in [a,b].$$

This theorem is important because polynomials possess excellent differentiation and integration properties: both the derivative and the integral of a polynomial are again polynomials. Another interpretation of Theorem 4.1 is that, given a continuous function on a closed and bounded interval, there exists a polynomial that can be made as "close" to the given function as desired.

In analysis, there is another important form of polynomial approximation, namely Taylor's theorem.

Theorem 4.2 (Taylor's Theorem)). [1, Theorem 6.4.1] *Suppose $f \in \mathcal{C}^n[a,b]$, $f^{(n+1)}$ exists on $[a,b]$, and $x_0 \in [a,b]$. Then for every $x \in [a,b]$ there exists a number $\xi(x) \in [x_0, x]$ such that*

$$f(x) = P_n(x) + R_n(x),$$

where

$$P_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k$$

and

$$R_n(x) = \frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0)^{n+1}.$$

There are two important issues with Taylor approximation:

Figure 4.3: Brook Taylor (18 August 1685 – 29 December 1731).

1. The higher-order derivatives of $f(x)$ must be known.
2. The approximation is *local*; it is highly accurate near x_0, but its accuracy deteriorates as we move away from x_0. For example, the Taylor series expansion of $\exp(x)$ about $x = 0$ becomes increasingly inaccurate as $|x|$ grows (see Fig. 4.4).

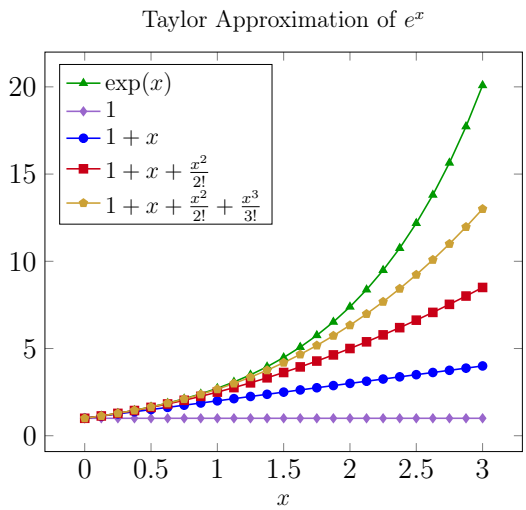

Figure 4.4: Taylor polynomials for the exponential function expanded about $x = 0$.

Nevertheless, Taylor's theorem remains a powerful theoretical tool and plays a central role in the derivation of numerical methods as well as in error estimation.

4.1 Lagrange Interpolation

We begin with Lagrange interpolation. The formula was first published by Waring in 1779, rediscovered by Euler in 1783, and later published by Lagrange in 1795.

Let us start with a basic example involving two data points (x_0, f_0) and (x_1, f_1). We define the functions

$$\mathbb{L}_0(x) = \frac{x - x_1}{x_0 - x_1} \qquad \text{and} \qquad \mathbb{L}_1(x) = \frac{x - x_0}{x_1 - x_0}. \tag{4.1}$$

Then the linear interpolating polynomial passing through these points is given by

$$p_1(x) = \mathbb{L}_0(x) f_0 + \mathbb{L}_1(x) f_1 = \frac{x - x_1}{x_0 - x_1} f_0 + \frac{x - x_0}{x_1 - x_0} f_1.$$

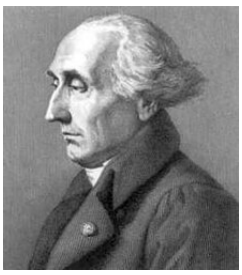

Figure 4.5: Joseph-Louis Lagrange (25 January 1736 – 10 April 1813).

Since $\mathbb{L}_0(x_0) = 1$, $\mathbb{L}_0(x_1) = 0$, $\mathbb{L}_1(x_0) = 0$, and $\mathbb{L}_1(x_1) = 1$, we immediately obtain $p_1(x_0) = f_0$ and $p_1(x_1) = f_1$. This polynomial is called the *Lagrange linear interpolating polynomial*. Moreover, this interpolating polynomial is unique. Figure 4.6 illustrates $\mathbb{L}_0(x)$ and $\mathbb{L}_1(x)$ for $x_0 = 0$ and $x_1 = 1$.

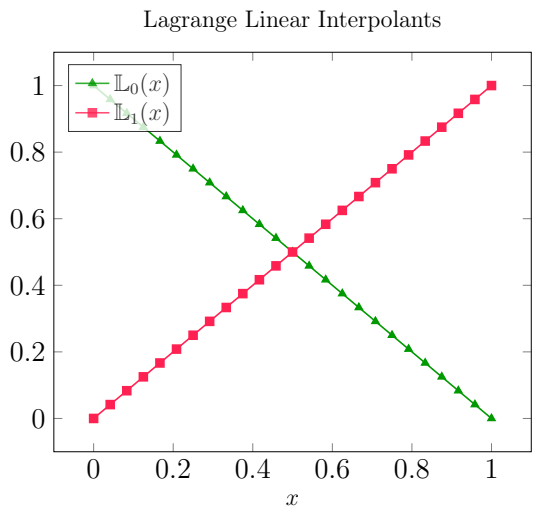

Figure 4.6: Lagrange linear basis polynomials for $x_i = \{0, 1\}$.

We now generalize this construction. Suppose we are given a set of data points $\{(x_i, f_i)\}_{i=0}^n$. For each $i = 0, 1, \ldots, n$, we construct a function $\mathbb{L}_{n,i}(x)$ such that

$$\mathbb{L}_{n,i}(x_k) = \delta_{ik} \qquad \text{for } k = 0, \ldots, n.$$

Motivated by Eq. (4.1), the general form of these functions is

$$\mathbb{L}_{n,i}(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \left(\frac{x - x_j}{x_i - x_j} \right).$$

The Lagrange interpolating polynomial is then defined as

$$p_n^{\mathbb{L}}(x) = \sum_{i=0}^{n} f_i \, \mathbb{L}_{n,i}(x). \tag{4.2}$$

When the degree is clear from the context, we write $\mathbb{L}_{n,i}(x)$ simply as $\mathbb{L}_i(x)$. The functions $\mathbb{L}_{n,i}(x)$ are called the *Lagrange basis polynomials of degree n*. Figure 4.7 shows these polynomials for equally spaced points.

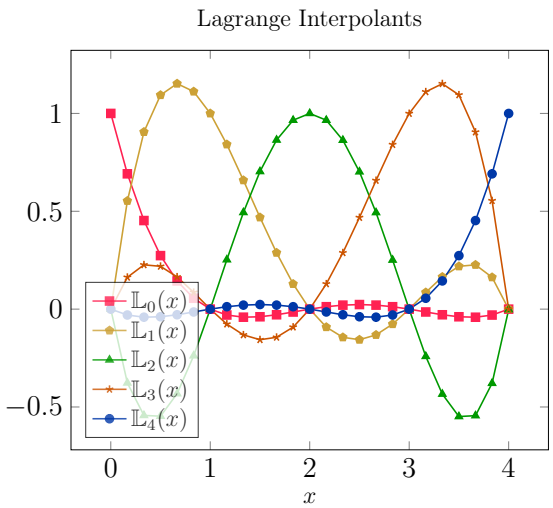

Figure 4.7: Lagrange basis polynomials defined over $x_i = 0, 1, 2, 3, 4$.

Example 4.1. Consider the data points $\{(x_i, f_i)\}_{i=0}^{2}$, where

$$x_0 = 0, \quad x_1 = 1, \quad x_2 = 2,$$

and

$$f_0 = 3, \quad f_1 = 0, \quad f_2 = 8.$$

Since we are given three data points, the interpolating polynomial will be of degree at most 2.

1. **Computation of $\mathbb{L}_0(x)$:**

$$\mathbb{L}_0(x) = \prod_{\substack{i=0 \\ i\neq 0}}^{2} \frac{x - x_i}{x_0 - x_i} = \frac{(x-1)(x-2)}{(0-1)(0-2)} = \frac{1}{2}(x-1)(x-2).$$

2. **Computation of $\mathbb{L}_1(x)$:**

$$\mathbb{L}_1(x) = \prod_{\substack{i=0 \\ i\neq 1}}^{2} \frac{x - x_i}{x_1 - x_i} = \frac{x(x-2)}{(1-0)(1-2)} = -x(x-2).$$

3. **Computation of $\mathbb{L}_2(x)$:**

$$\mathbb{L}_2(x) = \prod_{\substack{i=0 \\ i\neq 2}}^{2} \frac{x - x_i}{x_2 - x_i} = \frac{x(x-1)}{(2-0)(2-1)} = \frac{1}{2}x(x-1).$$

The Lagrange interpolating polynomial is therefore

$$\begin{aligned} p_2^{\mathbb{L}}(x) &= f_0\mathbb{L}_0(x) + f_1\mathbb{L}_1(x) + f_2\mathbb{L}_2(x) \\ &= \frac{3}{2}(x-1)(x-2) + 4x(x-1). \end{aligned}$$

We now state a few important remarks concerning Lagrange interpolation:

1. Equation (4.2) defines a linear map from the data space $\mathbb{R}^{n+1}$ to the space of polynomials $\mathbb{P}_n$.

2. The Lagrange interpolant can be extended to any continuous function $f(x)$ by defining

$$p_n^{\mathbb{L}} f(x) = \sum_{i=0}^{n} f(x_i)\, \mathbb{L}_i(x).$$

3. The operator $p_n^{\mathbb{L}}$ is a projection, that is, $p_n^{\mathbb{L}} q = q$ for all $q \in \mathbb{P}_n$.

We now present a theorem describing the error associated with Lagrange interpolation.

Theorem 4.3. *Suppose $\{x_0, x_1, \ldots, x_n\}$ are distinct points in the interval $[a, b]$ and $f \in \mathcal{C}^{n+1}[a, b]$. Then for each $x \in [a, b]$ there exists a number $\xi(x) \in (a, b)$ such that*

$$f(x) = p_n^{\mathbb{L}}(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!} \prod_{i=0}^{n} (x - x_i), \tag{4.3}$$

where $p_n^{\mathbb{L}}(x)$ is defined by Eq. (4.2).

Note that the error term resembles that of Taylor's theorem, but here it depends on all interpolation points rather than being centered at a single point.

Algorithm 11 Lagrange Interpolation

Given: Data points $\{(x_i, f_i)\}_{i=0}^{n}$, evaluation point x_{eval}
Find: Interpolated value $p_n^{\mathbb{L}}(x_{\text{eval}})$
  Step 1: Compute Lagrange basis functions at x_{eval}
  for $i = 0$ **to** n **do**
    $\mathbb{L}_i(x_{\text{eval}}) \leftarrow 1$
    for $j = 0$ **to** n **do**
      if $j \neq i$ **then**
        $\mathbb{L}_i(x_{\text{eval}}) \leftarrow \mathbb{L}_i(x_{\text{eval}}) \dfrac{x_{\text{eval}} - x_j}{x_i - x_j}$
      end if
    end for
  end for
  Step 2: Evaluate the Lagrange interpolating polynomial
  $p_n^{\mathbb{L}}(x_{\text{eval}}) \leftarrow 0$
  for $i = 0$ **to** n **do**
    $p_n^{\mathbb{L}}(x_{\text{eval}}) \leftarrow p_n^{\mathbb{L}}(x_{\text{eval}}) + f_i\, \mathbb{L}_i(x_{\text{eval}})$
  end for
  return $p_n^{\mathbb{L}}(x_{\text{eval}})$

4.1.1 Drawbacks

The Lagrange interpolant suffers from certain drawbacks. The first drawback concerns its *computational complexity.*

To evaluate the interpolant at a point x, consider an individual Lagrange basis polynomial of degree n,

$$\mathbb{L}_i(x) = \prod_{\substack{j=0 \\ j\neq i}}^{n} \frac{x - x_j}{x_i - x_j}.$$

The Lagrange interpolating polynomial is given by

$$p_n^{\mathbb{L}}(x) = f_0\mathbb{L}_0(x) + f_1\mathbb{L}_1(x) + \cdots + f_n\mathbb{L}_n(x).$$

To compute each $\mathbb{L}_i(x)$, we require $\mathcal{O}(n)$ multiplications. Since there are $(n+1)$ such basis polynomials, the total cost is $\mathcal{O}(n^2+n)$. The final evaluation of $p_n^{\mathbb{L}}(x)$ involves an additional $\mathcal{O}(n)$ multiplications and additions. Therefore, the overall computational complexity is $\mathcal{O}(n^2)$, which is undesirable when compared to linear complexity $\mathcal{O}(n)$.

Another major drawback is that if a new data point (x_{n+1}, f_{n+1}) is added, the entire interpolation must be recomputed from scratch.

Despite these drawbacks, Lagrange interpolation has certain advantages. For instance, the computation of the basis polynomials $\{\mathbb{L}_i(x)\}_{i=0}^n$ is independent of the function values f_i. Moreover, the formulation does not depend on the ordering of the interpolation nodes.

4.1.2 Runge Phenomenon

In 1901, Carl David Tolmé Runge observed that when approximating the function

$$f(x) = \frac{1}{1+25x^2}, \qquad x \in [-1, 1],$$

using polynomial interpolation at equally spaced points, large oscillations occur near the endpoints of the interval (see Fig. 4.9). This effect is known as the *Runge phenomenon*, and the above function is referred to as the *Runge function*.

Figure 4.8: Carl David Tolmé Runge (30 August 1856 – 3 January 1927).

To understand this behavior, consider the interpolation error from Theorem 4.3,

$$f(x) - p_n^{\mathbb{L}}(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}\prod_{i=0}^{n}(x - x_i), \qquad \xi \in (-1, 1).$$

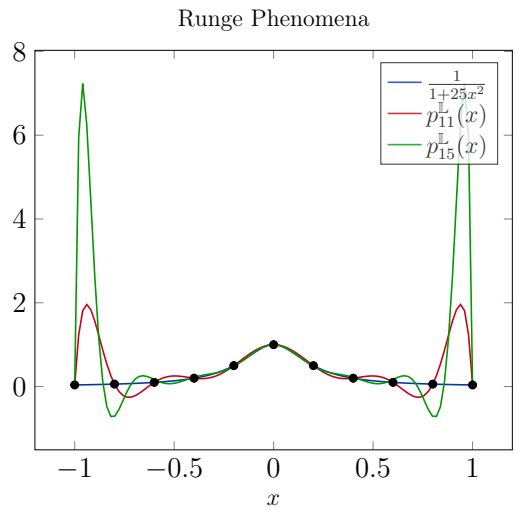

Figure 4.9: Runge phenomenon for the function $1/(1+25x^2)$. The polynomial $p_{11}^{\mathbb{L}}(x)$ is computed using 11 equally spaced points (shown as dots), while $p_{15}^{\mathbb{L}}(x)$ uses 15 points.

Taking the maximum over $[-1, 1]$, we obtain

$$\max_{-1\le x\le 1} |f(x) - p_n^{\mathbb{L}}(x)| \le \max_{-1\le x\le 1} \left| \frac{f^{(n+1)}(\xi)}{(n+1)!} \right| \max_{-1\le x\le 1} \prod_{i=0}^{n} |x - x_i|.$$

It can be shown (though the proof is nontrivial) that for equally spaced points with spacing $h = 2/n$,

$$\max_{-1\le x\le 1} \prod_{i=0}^{n} |x - x_i| \le h^{n+1} n!.$$

Furthermore, for the Runge function, the $(n+1)^{\text{th}}$ derivative can be bounded by

$$|f^{(n+1)}(x)| \le 5^{n+1}(n+1)!.$$

Combining these estimates yields

$$\max_{-1\le x\le 1} |f(x) - p_n^{\mathbb{L}}(x)| \gtrsim \left(\frac{10}{n}\right)^{n+1} n!,$$

which diverges as $n \to \infty$. This explains the increasingly poor approximation near the endpoints despite increasing the number of interpolation points.

One way to mitigate the Runge phenomenon is to use non-uniform grids with nodes clustered near the endpoints. For further details, see the review article by Berrut and Trefethen [2].

4.2 Newton Divided Difference Interpolation

In Sec. 4.1, we observed that Lagrange interpolation suffers from a computational complexity of $\mathcal{O}(n^2)$ for evaluation. We now introduce another interpolation method that overcomes this drawback, known as the *Newton Divided Difference* method.

Let $p_n(x)$ be a polynomial interpolating the data points $\{(x_i, f_i)\}_{i=0}^n$. One way of expressing such a polynomial is

$$p_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n \prod_{i=0}^{n-1}(x - x_i), \tag{4.4}$$

for appropriate constants $\{a_i\}_{i=0}^n$. The question is how to determine these coefficients.

At $x = x_0$, we have $p_n(x_0) = f_0$, which implies $a_0 = f_0$. Similarly, at $x = x_1$, we obtain $p_n(x_1) = f_1$, which gives

$$a_1 = \frac{f_1 - a_0}{x_1 - x_0} = \frac{f_1 - f_0}{x_1 - x_0}.$$

Proceeding in this manner quickly becomes cumbersome. To systematize the computation of the coefficients, we introduce the concept of *divided differences* (DD).

The zeroth divided difference of a function $f(x)$ at x_i is defined as

$$f[x_i] = f(x_i) = f_i.$$

Higher-order divided differences are defined recursively as follows:

- The first divided difference with respect to x_i and x_{i+1} is

$$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}. \tag{4.5}$$

- The second divided difference with respect to x_i, x_{i+1}, x_{i+2} is

$$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}. \tag{4.6}$$

- The kth divided difference with respect to $x_i, x_{i+1}, \ldots, x_{i+k}$ is

$$f[x_i, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}. \tag{4.7}$$

Using this notation, the polynomial defined in Eq. (4.4) can be written as

$$p_n^{\mathbb{N}}(x) = f[x_0] + \sum_{k=1}^{n} f[x_0, x_1, \ldots, x_k](x - x_0)(x - x_1)\ldots(x - x_{k-1}), \tag{4.8}$$

where $p_n^{\mathbb{N}}(x)$ denotes the Newton divided difference interpolating polynomial.

For illustration, the divided difference table corresponding to four data points is shown in Table 4.1.

$\boldsymbol{x}$	$\boldsymbol{f(x)} = \mathbf{0}^{\text{th}}$ DD	$\mathbf{1}^{\text{st}}$ DD	$\mathbf{2}^{\text{nd}}$ DD	$\mathbf{3}^{\text{rd}}$ DD
x_0	$f[x_0]$			
		$f[x_0, x_1]$		
x_1	$f[x_1]$		$f[x_0, x_1, x_2]$	
		$f[x_1, x_2]$		$f[x_0, x_1, x_2, x_3]$
x_2	$f[x_2]$		$f[x_1, x_2, x_3]$	
		$f[x_2, x_3]$		
x_3	$f[x_3]$			

Table 4.1: Divided difference table for four points x_0, x_1, x_2, x_3.

Example 4.2. Consider the data points $\{(x_i, f_i)\}_{i=0}^{2}$ given by

$$x_0 = 0, \quad x_1 = 1, \quad x_2 = 2,$$

with corresponding function values

$$f_0 = 3, \quad f_1 = 0, \quad f_2 = 8.$$

Since three data points are given, the interpolating polynomial will be of degree at most 2.

Step 1: Compute divided differences
The zeroth divided differences are

$$f[x_0] = 3, \qquad f[x_1] = 0, \qquad f[x_2] = 8.$$

The first divided differences are

$$f[x_0, x_1] = \frac{f_1 - f_0}{x_1 - x_0} = \frac{0-3}{1-0} = -3,$$

$$f[x_1, x_2] = \frac{f_2 - f_1}{x_2 - x_1} = \frac{8-0}{2-1} = 8.$$

The second divided difference is

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{8-(-3)}{2-0} = \frac{11}{2}.$$

Step 2: Construct the Newton interpolating polynomial
Using the Newton divided difference formula,

$$p_2^{\mathbb{N}}(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1),$$

we obtain

$$p_2^{\mathbb{N}}(x) = 3 - 3(x - 0) + \frac{11}{2}(x - 0)(x - 1).$$

Simplifying,

$$\begin{aligned} p_2^{\mathbb{N}}(x) &= 3 - 3x + \frac{11}{2}x(x-1) \\ &= 8x(x-1). \end{aligned}$$

Thus, the Newton interpolating polynomial coincides with the Lagrange interpolating polynomial obtained earlier.

While the Lagrange interpolating polynomial uses the basis functions $\mathbb{L}_{n,i}(x)$, the Newton divided difference method employs a different polynomial basis. This basis is defined by

$$\omega_i(x) = \prod_{k=0}^{i-1}(x - x_k), \qquad i \geq 1, \quad \text{and} \quad \omega_0(x) = 1,$$

as illustrated in Fig. 4.10.

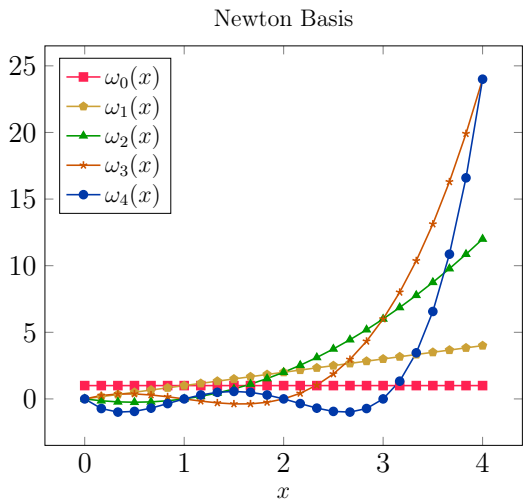

Figure 4.10: Newton basis polynomials defined over $x_i = 0, 1, 2, 3, 4$.

We conclude with a result that provides an explicit representation of the highest-order divided difference.

Theorem 4.4. *For distinct points $x_0, \ldots, x_n$, the nth divided difference satisfies*

$$f[x_0, x_1, \ldots, x_n] = \sum_{k=0}^{n} \frac{f(x_k)}{\prod_{i \neq k}(x_k - x_i)},$$

with $f[x_0] = f(x_0)$ for $n = 0$.

The algorithm for computing the Newton divided difference interpolation is presented in Algorithm 12.

Algorithm 12 Newton Interpolation

Given: Data points $\{(x_i, f_i)\}_{i=0}^{n}$, evaluation point x_{eval}
Find: Interpolated value $p_n^{\mathbb{N}}(x_{\text{eval}})$
  Step 1: Construct divided difference table
  Initialize DD as a zero matrix of size $(n+1) \times (n+1)$
  for $i = 0$ **to** n **do**
    $\text{DD}_{i,0} \leftarrow f_i$
  end for
  for $j = 1$ **to** n **do**
    for $i = 0$ **to** $n - j$ **do**
      $\text{DD}_{i,j} \leftarrow \dfrac{\text{DD}_{i+1,j-1} - \text{DD}_{i,j-1}}{x_{i+j} - x_i}$
    end for
  end for
  Step 2: Evaluate Newton interpolating polynomial at x_{eval}
  $p_n^{\mathbb{N}}(x_{\text{eval}}) \leftarrow \text{DD}_{0,0}$
  for $k = 1$ **to** n **do**
    $\omega \leftarrow 1$
    for $j = 0$ **to** $k - 1$ **do**
      $\omega \leftarrow \omega\,(x_{\text{eval}} - x_j)$
    end for
    $p_n^{\mathbb{N}}(x_{\text{eval}}) \leftarrow p_n^{\mathbb{N}}(x_{\text{eval}}) + \text{DD}_{0,k}\,\omega$
  end for
  return $p_n^{\mathbb{N}}(x_{\text{eval}})$

4.2.1 Computational Complexity

The Newton interpolating polynomial can be rewritten in a nested (Horner-like) form as

$$\begin{aligned} p_n^{\mathbb{N}}(x) &= a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1)\ldots(x - x_{n-1}) \\ &= a_0 + (x - x_0)\Big[a_1 + (x - x_1)\big(a_2 + \cdots + (x - x_{n-2})(a_{n-1} + a_n(x - x_{n-1}))\big)\Big]. \end{aligned}$$

In this form, each stage of the evaluation requires one multiplication and one addition. Since there are n stages, the total number of operations is approximately $2n$, which is $\mathcal{O}(n)$. In contrast, evaluating the Lagrange interpolating polynomial requires $\mathcal{O}(n^2)$ operations.

Another important advantage of Newton interpolation over Lagrange interpolation is that it is easy to update the divided difference table when a new data point is added. In this case, no recomputation from scratch is required; only a single new column of divided differences needs to be computed.

4.3 Summary

In this chapter, we studied *polynomial interpolation*, which plays a central role in numerical analysis when approximating functions from discrete data. Interpolation arises naturally

in many engineering and scientific applications, including data fitting, numerical integration, numerical differentiation, and the solution of differential equations. From a computational perspective, the choice of interpolation method has a significant impact on efficiency, accuracy, and stability.

We began by introducing the interpolation problem: given a set of data points $\{(x_i, f_i)\}_{i=0}^n$, find a polynomial $p_n(x)$ such that

$$p_n(x_i) = f_i, \qquad i = 0, 1, \ldots, n.$$

We discussed the theoretical foundation of polynomial approximation and motivated interpolation through results from analysis.

1. **Polynomial Approximation and Motivation**
   - The *Weierstrass Approximation Theorem* was introduced to establish the existence of polynomial approximations for continuous functions on closed and bounded intervals.
   - Taylor's theorem was discussed as a local polynomial approximation technique, highlighting its strengths in analysis and error estimation, as well as its limitations due to locality and dependence on higher derivatives.
2. **Lagrange Interpolation**
   - The *Lagrange interpolating polynomial* was derived using Lagrange basis polynomials and shown to be the unique polynomial interpolating the given data points.
   - An explicit formula for the interpolation error was obtained, revealing the dependence on the $(n+1)^{\text{th}}$ derivative of the function and the distribution of interpolation nodes.
   - We observed that Lagrange interpolation requires $\mathcal{O}(n^2)$ operations for evaluation and must be recomputed entirely if new data points are added.
   - The *Runge phenomenon* was studied as a warning example, demonstrating that interpolation at equally spaced points can lead to large oscillations near the endpoints of the interval.
3. **Newton Interpolation and Divided Differences**
   - The *Newton divided difference* formulation was introduced as an alternative representation of the interpolating polynomial.
   - The divided difference table provides a systematic and efficient way to compute the interpolation coefficients.
   - Using a nested (Horner-like) form, the Newton interpolating polynomial can be evaluated in $\mathcal{O}(n)$ operations.
   - A major advantage of Newton interpolation is that new data points can be incorporated easily by extending the divided difference table, without recomputing the entire polynomial.

Beyond This Chapter:

- While global polynomial interpolation provides a powerful theoretical framework, it may suffer from numerical instability and poor approximation properties for large data sets.
- The choice of interpolation nodes plays a crucial role in accuracy, motivating the use of non-uniform grids such as Chebyshev points.
- Piecewise polynomial methods, such as spline interpolation, offer improved stability and accuracy and are widely used in practical computations.
- These ideas will form the basis for subsequent chapters on numerical differentiation, numerical integration, and the numerical solution of differential equations.

List of Notation

Notation	Description
$\sum_{n=1}^{\infty}$	Summation of Numbers
x_{10}	Base 10 Number
x_2	Base 2 Number
$\mathbb{Z}$	Set of Integers
$\mathcal{C}[a,b]$	Set of continuous functions defined on $[a,b]$
$\mathcal{C}^{\infty}[a,b]$	Set of infinitely differentiable functions defined on $[a,b]$
$\mathbb{R}$	Set of Real Numbers
$\mathbb{N}$	Set of Natural Numbers

Table 4.2: List of Mathematical Notations

List of Algorithms

1 Bisection Method 22
2 Fixed Point Iteration Method 30
3 Newton–Raphson Method 33
4 Gaussian Elimination with Partial Pivoting 49
5 Gauss–Jordan Elimination 52
6 LU Decomposition 55
7 $\mathrm{LDL}^\top$ Decomposition 59
8 Cholesky Decomposition 61
9 Jacobi Iteration Method 67
10 Gauss–Seidel Iteration Method 69
11 Lagrange Interpolation 82
12 Newton Interpolation 88

Index

Absolute Error, 13
Augmented Matrix, 45

Band Matrix, 44
Binary Number System, 9
Bisection Method, 20

Characteristic, 11
Cholesky Decomposition, 60
Chopping, 13
Condition Number, 73
Conditioning, 16

Decimal Number System, 9
Diagonally Dominant Matrix, 56

Exponent, 11

Fixed Point, 27
Fixed-Point Iteration, 28
Frobenius Norm, 63

Gauss Seidel Method, 67
Gauss–Jordan Algorithm, 50
Gaussian Elimination, 44

Intermediate Value Theorem, 20

Jacobi Method, 64

Lagrange Interpolation, 79
$\mathrm{LDL}^\top$ Decomposition, 56
Leading Principal Sub-Matrix, 58
LU Decomposition, 51

Mantissa, 11
Matrix Norm, 63

Newton Divided Differences, 84
Numerical Instability, 15

Order of Convergence, 36
Overflow, 12

Pivot, 48
Positive Definite Matrix, 57
Preconditioning, 74

Regular Falsi, 25
Relative Error, 13
Round-off Errors, 13
Rounding, 13
Runge Function, 83
Runge Phenomenon, 83

Secant Method, 36
Significant Digits, 14
Spectral Radius, 70
Symmetric Positive Definite, 57

Taylor's Theorem, 78
Truncation Error, 17

Underflow, 12

Vector Norm, 62

Weierstrass Approximation Theorem, 78

Bibliography

[1] R. G. Bartle and D. R. Sherbert. *Introduction to real analysis*. Second. John Wiley & Sons, Inc., New York, 1992, pp. xii+404. ISBN: 0-471-51000-9.

[2] J.-P. Berrut and L. N. Trefethen. "Barycentric Lagrange interpolation". In: *SIAM Rev.* 46.3 (2004), pp. 501–517. ISSN: 0036-1445,1095-7200. DOI: 10.1137/S0036144502417715. URL: https://doi.org/10.1137/S0036144502417715.

[3] J. D. Faires and R. Burden. *Numerical methods*. Second. With 1 IBM-PC floppy disk (3.5 inch; HD). Brooks/Cole Publishing Co., Pacific Grove, CA, 1998, pp. xii+594. ISBN: 0-534-35187-5.

[4] "IEEE Standard for Binary Floating-Point Arithmetic". In: *ANSI/IEEE Std 754-1985* (1985), pp. 1–20. DOI: 10.1109/IEEESTD.1985.82928.

[5] "IEEE Standard for Floating-Point Arithmetic". In: *IEEE Std 754-2008* (2008), pp. 1–70. DOI: 10.1109/IEEESTD.2008.4610935.

[6] G. Strang. *Linear algebra and its applications*. Academic Press [Harcourt Brace Jovanovich, Publishers], New York-London, 1976, pp. xi+374.